SkillNavora Academy · Master Freelancing

Master Freelancing Lesson Guide

Lesson-by-lesson guide for Departments 1 to 7, with the prompt and tool handbook

80 lessons · tested code samples · practice tasks with answers · Official site: www.skillnavora.in

Tools, prices, model names and rules change. Official links were not checked live in this draft; confirm before relying on them.

Contents

1. How to use this lesson guide
2. Development and IT
3. AI and Automation
4. Design and Creative
5. Marketing
6. Data Analytics and Data Science
7. Admin and Business Support
8. Writing and Content
9. Master AI video prompting system

START HERE

How to use this lesson guide

This guide follows the lesson numbering for the seven departments and links each lesson to a practice task. Use it with the skill textbooks, which give the professions, portfolio projects and freelancing steps.

How the pieces fit

Part	Use it to
This lesson guide	Learn each topic in order and practise it
Skill textbooks (one per department)	Build a client-ready portfolio project and learn the freelancing side of each profession
Freelance Business and Agency book	Find clients, price, contract and get paid
Prompt and tool handbook (end of this book)	Reuse tested prompts, workflows and tool lists

VERIFICATION NOTE

Tools, plans, prices, model names and platform rules change often. Links point to official sites that were not checked live in this draft. Confirm every price, limit and policy on the official site before relying on it. Samples use fictional data. This book is learning material, not legal, tax or financial advice, and it does not promise jobs or income.

Lesson index (80 lessons)

Dept	Lesson	Title
1	1.1	Foundations of full-stack architecture
1	1.2	Frontend engineering (React and Next.js)
1	1.3	Backend engineering and databases
1	1.4	AI-assisted rapid development (vibe coding)
1	1.5	Deployment and DevOps
1	2.1	WordPress core and custom theme development
1	2.2	Page builders and dynamic content
1	2.3	Automation integration (WordPress and n8n)
1	2.4	Speed optimisation and hardening
1	3.1	HTML5 semantic structure and WCAG accessibility
1	3.2	Advanced CSS3
1	3.3	Modern JavaScript (ES6+)
1	4.1	Liquid templating engine
1	4.2	Custom sections and Shopify CLI workflows

1	4.3	Storefront API and headless commerce
1	5.1	Cross-platform engineering with React Native and Flutter
1	5.2	Native device APIs
1	5.3	App store optimisation and deployment
1	6.1	Webflow box model, responsive layouts and Client-First naming
1	6.2	Webflow CMS, dynamic filtering and custom code
1	7.1	React hooks
1	7.2	SSR vs SSG in Next.js
1	8.1	Cart logic, checkout, PCI-DSS and multi-currency
1	8.2	Payment gateway integration
1	9.1	Next.js App Router, server actions, edge functions and middleware
1	10.1	Visual database modelling, privacy rules, API Connector and plugins
2	1.1	Image-to-video and text-to-video platforms
2	1.2	Prompt engineering for consistent scenes and characters
2	1.3	Video production case studies and prompts
2	1.4	Credit optimisation and efficiency
2	1.5	Post-production and audio sync
2	2.1	OpenAI API and Anthropic SDK integration in Node.js and Python
2	2.2	Vector databases: Pinecone, ChromaDB and Weaviate
2	2.3	Retrieval-augmented generation (RAG) and document indexing
2	3.1	RAG-powered bots in Voiceflow, Botpress and Flowise
2	3.2	WhatsApp Business API integration via Twilio and Wati
2	4.1	Data preprocessing and model training (Pandas, NumPy, Scikit-Learn, PyTorch)
2	5.1	Multi-step scenarios, JSON parsing, array aggregators and webhooks
2	6.1	Self-hosting n8n with Docker on DigitalOcean or AWS
2	6.2	Custom JavaScript nodes, error workflows and API polling
2	7.1	AI-first coding workflows with Cursor, Replit Agent and Claude
2	8.1	System prompt optimisation, context window management and artifacts
2	9.1	Corporate AI audits, ROI models and technology stack mapping
3	1.1	Industry-specific logo systems
3	1.2	Tooling and workflow
3	1.3	Asset sourcing and typography
3	2.1	Principles of layout
3	2.2	Social media, ads and print collateral

3	3.1	Design systems in Figma
3	4.1	Timeline editing, J/L cuts, sound design and colour grading
3	5.1	Non-destructive editing, frequency separation and compositing
3	6.1	User research, wireframing, information architecture and usability testing
3	7.1	Slide master, data visualisation and pitch decks
3	8.1	Full brand identity systems
3	9.1	Polygon modelling, PBR texturing, lighting and rendering in Blender
4	1.1	Daily workflow and operations routine
4	1.2	Account onboarding and audit procedures
4	2.1	Keyword research, ad groups and match types
4	2.2	Conversion tracking, Performance Max and bidding
4	3.1	On-page optimisation
4	3.2	Technical audits, link building and Search Console
4	4.1	Campaign set-up for real estate, tech agencies and e-commerce
4	4.2	Budget optimisation and ROAS: audiences and the Conversions API
4	5.1	Deliverability: SPF, DKIM and DMARC
4	5.2	Outreach infrastructure at scale, safely
4	6.1	CRM integration, lead scoring and nurture funnels
4	7.1	B2B prospecting, outreach scripts and PPC landing pages
5	1.1	AI-powered analytics with Microsoft Copilot
5	2.1	EDA with Python, feature engineering and predictive models
5	3.1	ETL pipelines, data warehousing and indexing
5	4.1	Data modelling, star schemas, DAX and publishing
6	1.1	Executive support fundamentals
6	1.2	Remote SOP creation in Notion
6	2.1	Advanced data cleansing, verification and OCR
6	3.1	Double-entry accounting, chart of accounts, reconciliation and ageing
6	4.1	Scripting, gatekeepers, objections and calendar booking
7	1.1	Industry-specific ATS optimisation
7	2.1	Direct response copy, headlines and funnel scripts (PAS and AIDA)
7	3.1	Developer documentation, API references and knowledge base guides in Markdown
7	4.1	VSL scripts, YouTube outlines and e-book ghostwriting

DEPARTMENT 1

Development and IT

Lessons for Department 1. Each role has numbered lessons with what you learn, official tools, a practice task with an answer, and short questions. The matching skill textbook is the Master Freelancing Development and IT book.

1. Full-Stack Developer

A full-stack developer builds both what users see (frontend) and the server and database behind it (backend).

Lesson 1.1: Foundations of full-stack architecture

What you learn

- **Client-server model.** A client (browser or app) sends a request; a server processes it and sends a response. The server holds data and rules; the client shows them.
- **HTTP protocol.** Requests have a method (GET, POST, PUT, PATCH, DELETE), a path, headers and sometimes a body. Responses carry a status code (200 OK, 201 Created, 400 bad request, 401 not signed in, 404 not found, 500 server error), headers and a body.
- **RESTful APIs.** Resources have URLs (for example /users/1) and the HTTP method says the action. Responses are usually JSON. Each request carries what the server needs; the server keeps no hidden session between calls.
- **GraphQL.** One endpoint where the client asks for exactly the fields it needs. It reduces over-fetching but adds a schema, resolvers and caching to learn. REST is simpler for small APIs; GraphQL helps when many screens need different shapes of the same data.

Tools and resources (official sites, not checked live)

- MDN Web Docs: HTTP
- Postman documentation

ASSIGNMENT

Practice for lesson 1.1

Business problem: Prove you can read and make an HTTP exchange.

Steps

1. Run the tested server and client below.
2. Open the same kind of request in Postman against any public API.
3. Note the method, status code and content type of each response.

Expected output: A 200 response with a JSON body and a note of the three values.

SOLUTION (ANSWER KEY)

PYTHON

```
import threading, json, urllib.request
from http.server import BaseHTTPRequestHandler, HTTPServer
```

```
class H(BaseHTTPRequestHandler):
    def do_GET(self):
        body=json.dumps({"id":1,"name":"Asha"}).encode()
        self.send_response(200); self.send_header("Content-Type","application/json");
self.send_header("Content-Length",str(len(body))); self.end_headers(); self.wfile.write(body)
    def log_message(self,*a): pass
s=HTTPServer(("127.0.0.1",0),H); port=s.server_port
threading.Thread(target=s.serve_forever,daemon=True).start()
r=urllib.request.urlopen(f"http://127.0.0.1:{port}/users/1")
print(r.status, r.headers["Content-Type"], json.loads(r.read()))
assert r.status==200
s.shutdown()
```

The output shows status 200, content type application/json and the parsed body. In Postman you will see the same three items in the response panel.

Check yourself

1. Which status code means 'created'?
   Answer: 201.
2. When might GraphQL beat REST?
   Answer: When many clients need different subsets of the same data and over-fetching is a real cost.

Lesson 1.2: Frontend engineering (React and Next.js)

What you learn

- **DOM manipulation.** The DOM is the live tree of page elements. Plain JavaScript changes it directly; React describes the page from state and updates the DOM for you.
- **Component lifecycle.** A component mounts, updates when its state or props change, and unmounts. In function components, useEffect runs side effects after render and its cleanup runs before the next effect or on unmount.
- **State management.** Keep state as local as possible. Share it with context for small cases, Zustand for light global stores, or Redux Toolkit when many parts of a large app need predictable, traceable updates.

Tools and resources (official sites, not checked live)

- React documentation
- Next.js documentation
- Zustand
- Redux Toolkit

ASSIGNMENT

Practice for lesson 1.2

Business problem: Build a counter and a shared cart count.

Steps

1. Create a Next.js app.
2. Build a counter with useState.
3. Move the count into a Zustand store and show it in two components.
4. Confirm both update together.

Expected output: Two components that stay in sync through one store.

SOLUTION (ANSWER KEY)

A store created with `create()` exposes the count and an increment function. Both components read it with the same hook, so one update re-renders both.

Check yourself

1. What does useEffect cleanup do?
   Answer: Runs before the next effect or on unmount to release timers or subscriptions.
2. When is Redux Toolkit worth it?
   Answer: Large apps with many interacting parts needing predictable, traceable state changes.

Lesson 1.3: Backend engineering and databases

What you learn

- **Node.js and Express.** Node runs JavaScript on the server. Express adds routes and middleware: functions that run in order on each request (logging, auth, validation) before the route handler.
- **ORMs: Prisma and Drizzle.** An ORM maps database tables to code. Prisma uses a schema file and generates a typed client; Drizzle writes the schema in TypeScript and stays close to SQL. Both give migrations.
- **PostgreSQL.** A relational database: tables, keys, joins, constraints and transactions. Choose it when data has clear relationships and integrity matters.
- **MongoDB.** A document database that stores JSON-like documents. Choose it when records vary in shape; design around how the data is read.

Tools and resources (official sites, not checked live)

- Node.js
- Express
- Prisma
- Drizzle ORM
- PostgreSQL
- MongoDB

ASSIGNMENT

Practice for lesson 1.3

Business problem: Create a small tasks API.

Steps

1. Create a Postgres database and a tasks table.
2. Add an Express app with GET and POST /tasks.
3. Validate input before saving.
4. Test with Postman.

Expected output: Creating a task returns 201; listing returns the saved tasks.

SOLUTION (ANSWER KEY)

Use a parameterised query or the ORM client so user input never goes into SQL text. Return 400 with a message when the title is empty.

Check yourself

1. Why use parameterised queries?
   Answer: They keep user input from being run as SQL, which prevents SQL injection.
2. What is middleware?
   Answer: A function that runs before the route handler, such as auth or validation.

Lesson 1.4: AI-assisted rapid development (vibe coding)

What you learn

- **What it is.** Describing the app in words and letting an AI tool generate code. You still review, test and own the result.
- **Tools.** Lovable and Bolt generate whole web apps from prompts. Cursor is a code editor with an AI assistant that edits your own project.
- **Lovable to Supabase workflow.** 1. Write a prompt with the user, the screens and the data. 2. Generate the UI. 3. Connect the project to Supabase from the tool's integration settings. 4. Create tables and row-level security rules. 5. Test sign-in and a read and write. 6. Review generated code for secrets and open access.
- **Safety.** Never paste secrets into prompts. Check access rules by testing as a signed-out user and as a different user.

Tools and resources (official sites, not checked live)

- Lovable
- Bolt
- Cursor
- Supabase documentation

ASSIGNMENT

Practice for lesson 1.4

Business problem: Build a feedback form with saved responses.

Steps

1. Prompt Lovable for a form with name, rating and comment.
2. Connect Supabase.
3. Create a table and a rule that allows inserts but not public reads.
4. Test from a signed-out browser.

Expected output: Submissions save; the public cannot read other people's feedback.

SOLUTION (ANSWER KEY)

The key test is access: an insert-only rule lets visitors send feedback while reads stay limited to the owner.

Check yourself

1. Who is responsible for AI-generated code?
   Answer: You are; review and test it.
2. What should you test after connecting a database?
   Answer: Access rules, from signed-out and different-user views.

Lesson 1.5: Deployment and DevOps

What you learn

- **CI/CD pipelines.** Continuous integration runs tests on every change; continuous delivery deploys passing builds. GitHub Actions is a common way to define them.
- **Vercel and Render.** Vercel suits frontends and Next.js; Render runs servers, databases and background workers.
- **Docker.** A container packages the app with its dependencies so it runs the same everywhere. A Dockerfile lists the steps.
- **AWS S3 asset hosting.** S3 stores files such as images. Keep buckets private by default and serve public assets deliberately, often through a CDN.

Tools and resources (official sites, not checked live)

- GitHub Actions
- Vercel
- Render
- Docker
- Amazon S3

ASSIGNMENT

Practice for lesson 1.5

Business problem: Ship the tasks API.

Steps

1. Write a Dockerfile.
2. Add a workflow that runs tests on push.
3. Deploy to Render.
4. Store environment variables in the host, not in the repository.

Expected output: A public URL that returns your tasks and a green workflow run.

SOLUTION (ANSWER KEY)

Secrets live in the platform's environment settings. The workflow fails the build if tests fail, so broken code never deploys.

Check yourself

1. Why keep secrets out of the repo?
   Answer: Anyone with the code could use them.
2. What does CI do?
   Answer: Runs tests automatically on each change.

2. WordPress Developer

A WordPress developer builds and customises sites on the world's most used CMS.

Lesson 2.1: WordPress core and custom theme development

What you learn

- **PHP fundamentals.** Variables, arrays, functions and conditionals; WordPress runs on PHP.
- **Template hierarchy.** WordPress picks the template file by page type: single.php for a post, page.php for a page, archive.php for lists, falling back to index.php.
- **Hooks.** Actions run your code at set points (add_action). Filters change a value before it is used (add_filter).
- **wp-config security.** Keep wp-config.php outside public access where possible, use unique salts, disable file editing (DISALLOW_FILE_EDIT) and never commit credentials.

Tools and resources (official sites, not checked live)

- WordPress developer resources
- Theme handbook

ASSIGNMENT

Practice for lesson 2.1

Business problem: Create a child theme with one filter.

Steps

1. Create a child theme folder with style.css and functions.php.
2. Add a filter that appends a line to post titles.
3. Activate it on a test site.

Expected output: The filtered title appears on the site.

SOLUTION (ANSWER KEY)

Use add_filter('the_title', ...) and return the changed title. A child theme keeps edits safe when the parent updates.

Check yourself

1. Action vs filter?
   Answer: An action runs code; a filter returns a changed value.
2. Why a child theme?
   Answer: Updates to the parent do not erase your changes.

Lesson 2.2: Page builders and dynamic content

What you learn

- **Elementor Pro and Bricks.** Visual builders for layouts. Bricks tends to be lighter; Elementor has a larger ecosystem. Both need care to keep pages fast.
- **Advanced Custom Fields.** Adds structured fields (price, date, image) to posts and pages.
- **Custom Post Types.** Create content types such as Properties or Courses beyond Posts and Pages.

Tools and resources (official sites, not checked live)

- Elementor
- Bricks Builder
- ACF

ASSIGNMENT

Practice for lesson 2.2

Business problem: Build a Courses listing.

Steps

1. Register a Courses custom post type.
2. Add ACF fields for price and duration.
3. Show them in a builder template.

Expected output: A listing that updates when you add a course.

SOLUTION (ANSWER KEY)

The template reads the ACF fields dynamically, so new courses need no layout work.

Check yourself

1. Why use a custom post type?
   Answer: To keep different content types organised with their own fields.
2. What do ACF fields add?
   Answer: Structured data for each item.

Lesson 2.3: Automation integration (WordPress and n8n)

What you learn

- **REST API.** WordPress exposes content at /wp-json/. Create an application password to call it securely.
- **Webhook workflow.** 1. WooCommerce order created, webhook fires. 2. n8n Webhook node receives it. 3. Append a row to Google Sheets. 4. Send a Slack message. 5. Send a WhatsApp message through the Twilio API after checking the customer's consent. 6. Add an error workflow that alerts you on failure.
- **Reliability.** Verify webhook signatures, return 200 quickly and make the flow safe to run twice.

Tools and resources (official sites, not checked live)

- WordPress REST API
- n8n documentation
- Twilio WhatsApp

ASSIGNMENT

Practice for lesson 2.3

Business problem: Build the order alert flow.

Steps

1. Create a WooCommerce webhook for order creation.
2. Build the n8n flow above.
3. Place a test order.
4. Confirm a sheet row, a Slack message and a WhatsApp message.

Expected output: One test order produces all three outputs, once.

SOLUTION (ANSWER KEY)

Use the order ID as the key so a retry updates the same row instead of adding a duplicate.

Check yourself

1. Why verify webhook signatures?
   Answer: To confirm the request came from your store.
2. How do you avoid duplicates?
   Answer: Key on the order ID.

Lesson 2.4: Speed optimisation and hardening

What you learn

- **Redis caching.** An object cache keeps repeated database results in memory.
- **WP Rocket.** A caching plugin: page cache, file minification and lazy loading. Test after each setting.
- **Cloudflare CDN.** Serves static files from nearby locations and adds protection.
- **Security.** Keep core, themes and plugins updated; limit login attempts; use two-factor sign-in; take backups and test restoring them.

Tools and resources (official sites, not checked live)

- WP Rocket
- Cloudflare
- Redis

ASSIGNMENT

Practice for lesson 2.4

Business problem: Speed up a test site.

Steps

1. Measure with PageSpeed Insights.
2. Enable page cache and compression.
3. Add the CDN.
4. Measure again.

Expected output: A before and after score with the changes listed.

SOLUTION (ANSWER KEY)

Change one setting at a time and re-test so you know what helped and what broke.

Check yourself

1. Why test restoring backups?
   Answer: A backup you cannot restore is no backup.
2. Why change one setting at a time?
   Answer: To find what caused a problem.

3. Web Developer (vanilla JS and modern web standards)

Core standards that every other web role builds on.

Lesson 3.1: HTML5 semantic structure and WCAG accessibility

What you learn

- **Semantic elements.** header, nav, main, section, article, footer give meaning to assistive tools and search engines.
- **WCAG basics.** Perceivable, operable, understandable, robust. Practical checks: text alternatives for images, contrast of at least 4.5:1 for body text, keyboard access to everything, visible focus, labelled form fields.
- **Testing.** Tab through the page, use a screen reader, and run an automated checker; automated tools catch only part of the issues.

Tools and resources (official sites, not checked live)

- MDN accessibility
- WCAG overview

ASSIGNMENT

Practice for lesson 3.1

Business problem: Fix an inaccessible form.

Steps

1. Add labels to every input.
2. Make the button a real button element.
3. Check contrast.
4. Tab through it.

Expected output: A form that works by keyboard and passes an automated check.

SOLUTION (ANSWER KEY)

A label tied to an input by `for` and `id` lets screen readers announce the field.

Check yourself

1. Why use main and nav?
   Answer: They give structure that assistive tools can jump to.
2. Do automated checks cover everything?
   Answer: No; test by keyboard too.

Lesson 3.2: Advanced CSS3

What you learn

- **Flexbox.** One-dimensional layout: rows or columns with alignment and spacing.
- **Grid.** Two-dimensional layout with rows and columns defined together.
- **CSS variables.** Custom properties such as --brand store design values once.
- **Animations.** Use transitions and keyframes on transform and opacity for smooth results; respect prefers-reduced-motion.
- **Tailwind CSS.** Utility classes written in the markup. Fast to build; keep components organised.

Tools and resources (official sites, not checked live)

- MDN CSS
- Tailwind CSS

ASSIGNMENT

Practice for lesson 3.2

Business problem: Build a responsive card grid.

Steps

1. Create a Grid that shows one column on phones and three on desktop.
2. Store colours in variables.
3. Add a hover transition.

Expected output: A grid that adapts and passes a phone-width check.

SOLUTION (ANSWER KEY)

Use `grid-template-columns: repeat(auto-fit, minmax(240px, 1fr))` for the adaptive columns.

Check yourself

1. Flexbox or Grid for a page layout with rows and columns?
   Answer: Grid.
2. Why prefers-reduced-motion?
   Answer: Some users get discomfort from motion.

Lesson 3.3: Modern JavaScript (ES6+)

What you learn

- **Async/await.** Writes promise code in a readable order. Always handle errors with try/catch.
- **Fetch API.** fetch returns a promise; check response.ok because HTTP errors do not reject.
- **Event loop.** Synchronous code runs first, then microtasks (promises), then timers and other tasks.
- **Modules.** import and export split code into files with clear dependencies.

Tools and resources (official sites, not checked live)

- MDN JavaScript

ASSIGNMENT

Practice for lesson 3.3

Business problem: Predict the order, then run it.

Steps

1. Read the code below.
2. Write down the output order.
3. Run it with Node.

Expected output: A to D in the order shown below.

SOLUTION (ANSWER KEY)

JAVASCRIPT

```
const order=[];
order.push('A: sync start');
setTimeout(()=>{order.push('D: timer (macrotask)');console.log(order.join('\n'));},0);
Promise.resolve().then(()=>order.push('C: promise (microtask)'));
(async()=>{await null; order.push('C2: after await (microtask)');})();
order.push('B: sync end');
```

Output order: A (sync), B (sync), C (promise microtask), C2 (after await), D (timer). Microtasks run before the timer even with a zero delay.

Check yourself

1. Why check response.ok?
   Answer: fetch does not reject on 404 or 500.
2. Which runs first: a promise callback or setTimeout 0?
   Answer: The promise callback.

4. Shopify Developer

Builds and customises Shopify stores and themes.

Lesson 4.1: Liquid templating engine

What you learn

- **Tags.** Control flow such as {% if %} and {% for %}.
- **Objects.** Data such as product, collection and cart, printed with {{ }}.
- **Filters.** Change values: {{ product.price | money }}.
- **Schema settings.** A JSON block in a section that gives merchants editable settings in the theme editor.

Tools and resources (official sites, not checked live)

- Liquid reference
- Shopify theme docs

ASSIGNMENT

Practice for lesson 4.1

Business problem: Show sale prices.

Steps

1. Loop over collection products.
2. Print title and price with the money filter.
3. Show a Sale label when compare_at_price is higher.

Expected output: A grid with Sale labels on the right products.

SOLUTION (ANSWER KEY)

Compare `product.compare_at_price > product.price` inside an if tag.

Check yourself

1. Tag vs object?
   Answer: A tag controls logic; an object holds data.
2. What does schema do?
   Answer: Defines settings merchants can edit.

Lesson 4.2: Custom sections and Shopify CLI workflows

What you learn

- **Sections.** Reusable theme blocks with their own markup, settings and blocks.
- **Shopify CLI.** Run the theme locally, push to a development theme and pull changes. Work on a copy, not the live theme.
- **Version control.** Keep the theme in Git so every change has a history.

Tools and resources (official sites, not checked live)

- Shopify CLI

ASSIGNMENT

Practice for lesson 4.2

Business problem: Create a promo banner section.

Steps

1. Create the section file with schema settings for text and link.
2. Preview with the CLI.
3. Add it to the home page in the editor.

Expected output: A banner the merchant can edit without code.

SOLUTION (ANSWER KEY)

Use schema settings of type text and url so the merchant controls the content.

Check yourself

1. Why work on a development theme?
   Answer: To avoid breaking the live store.
2. Why use Git?
   Answer: To track and undo changes.

Lesson 4.3: Storefront API and headless commerce

What you learn

- **Storefront API.** Read products and manage carts from your own front end using GraphQL.

- **Headless.** The front end is separate from Shopify's theme. You gain control and take on hosting and performance work.
- **Hydrogen and Next.js.** Hydrogen is Shopify's React framework for storefronts; Next.js also works with the Storefront API.

Tools and resources (official sites, not checked live)

- Storefront API
- Hydrogen

ASSIGNMENT

Practice for lesson 4.3

Business problem: Fetch products into Next.js.

Steps

1. Create a Storefront access token.
2. Query the first ten products.
3. Render a list.

Expected output: A page that lists live products.

SOLUTION (ANSWER KEY)

Keep the token in an environment variable and call the API from server code where possible.

Check yourself

1. When is headless worth it?
   Answer: When you need custom experiences a theme cannot give.
2. Where should tokens live?
   Answer: In environment variables.

5. Mobile Application Developer

Builds apps for phones.

Lesson 5.1: Cross-platform engineering with React Native and Flutter

What you learn

- **React Native.** JavaScript and React; renders native components.
- **Flutter.** Dart; draws its own interface with a consistent look across platforms.
- **Choosing.** Pick React Native if the team already knows React; pick Flutter for highly custom visuals and one rendering engine.

Tools and resources (official sites, not checked live)

- React Native
- Flutter

ASSIGNMENT

Practice for lesson 5.1

Business problem: Build a two-screen app.

Steps

1. Create a list screen and a detail screen.
2. Navigate between them.
3. Run on an emulator.

Expected output: A working list and detail flow.

SOLUTION (ANSWER KEY)

Pass the selected item to the detail screen through navigation parameters.

Check yourself

1. Which suits a team that knows React?
   Answer: React Native.
2. What is Flutter written in?
   Answer: Dart.

Lesson 5.2: Native device APIs

What you learn

- **Camera and GPS.** Ask permission in context and handle refusal gracefully.
- **Push notifications.** Register a device token and send through a push service; ask for consent and keep messages useful.
- **Offline storage.** Store data locally and sync when online; decide how conflicts are resolved.

Tools and resources (official sites, not checked live)

- Expo documentation

ASSIGNMENT

Practice for lesson 5.2

Business problem: Add offline notes.

Steps

1. Save notes locally.
2. Show them with no connection.
3. Sync when back online.

Expected output: Notes appear in flight mode.

SOLUTION (ANSWER KEY)

Write to local storage first, then queue a sync so the user never waits on the network.

Check yourself

1. When to ask for camera permission?
   Answer: At the moment the feature is used, with a reason.
2. Why sync rules?
   Answer: Two devices may edit the same data.

Lesson 5.3: App store optimisation and deployment

What you learn

- **ASO.** Title, subtitle, keywords, screenshots and ratings influence discovery. Be accurate.
- **Google Play Console.** Create the app, upload a signed bundle, complete the content and data-safety forms, test, then release.
- **Apple App Store.** Use an Apple Developer account, upload through App Store Connect, add privacy details and pass review.

Tools and resources (official sites, not checked live)

- Google Play Console help
- App Store Connect

ASSIGNMENT

Practice for lesson 5.3

Business problem: Prepare store listings.

Steps

1. Write the title and short description.
2. Plan five screenshots.
3. Fill in the privacy form honestly.

Expected output: A listing checklist ready to submit.

SOLUTION (ANSWER KEY)

Listings must describe what the app does and the data it collects. Rules and fees change; read the current policy.

Check yourself

1. Why be accurate in listings?
   Answer: Stores reject misleading apps.
2. What do reviewers check?
   Answer: Function, content and privacy.

6. Webflow Developer

Builds production sites visually, with clean structure.

Lesson 6.1: Webflow box model, responsive layouts and Client-First naming

What you learn

- **Box model.** Content, padding, border, margin. Webflow shows them in the style panel.
- **Responsive layouts.** Design for desktop then adjust breakpoints; test on real phones.
- **Client-First.** A naming system with consistent class names (for example section_ and u- utilities) so any teammate can edit the project.

Tools and resources (official sites, not checked live)

- Webflow University
- Client-First

ASSIGNMENT

Practice for lesson 6.1

Business problem: Build a landing section.

Steps

1. Create a section with a container.
2. Name the classes by the system.
3. Check tablet and phone breakpoints.

Expected output: A section that behaves on three sizes.

SOLUTION (ANSWER KEY)

Consistent names make the project easy to hand over.

Check yourself

1. What does Client-First solve?
   Answer: Messy, hard-to-maintain class names.
2. What do you test at the end?
   Answer: Every breakpoint.

Lesson 6.2: Webflow CMS, dynamic filtering and custom code

What you learn

- **CMS collections.** Fields, items and collection pages that share one template.
- **Dynamic filtering.** Filter lists by category or reference fields.
- **Custom code.** Add scripts in project settings or embeds, test them, and avoid loading unneeded libraries.

Tools and resources (official sites, not checked live)

- Webflow CMS

ASSIGNMENT

Practice for lesson 6.2

Business problem: Create a blog with categories.

Steps

1. Create Posts and Categories collections.
2. Link them.
3. Show a filtered list.

Expected output: A blog with a category filter.

SOLUTION (ANSWER KEY)

Use a reference field from Posts to Categories and filter the collection list.

Check yourself

1. What is a collection page?
   Answer: One template that renders every item.
2. Why minimise custom scripts?
   Answer: They slow pages and may break.

7. Frontend Developer (React.js focus)

Lesson 7.1: React hooks

What you learn

- **useState.** Local state; update with the setter, never by changing the value directly.
- **useEffect.** Side effects after render; list dependencies carefully and clean up.
- **useContext.** Share values such as theme or user without passing props down every level.
- **useMemo and useCallback.** Cache a computed value or a function between renders. Use them when you have measured a problem, not by default.

Tools and resources (official sites, not checked live)

- React hooks reference

ASSIGNMENT

Practice for lesson 7.1

Business problem: Fix a stale-data bug.

Steps

1. Write an effect that fetches by id.
2. Add the id to the dependency list.
3. Add a cleanup that ignores old responses.

Expected output: The shown data always matches the selected id.

SOLUTION (ANSWER KEY)

Without the id dependency the effect would not re-run. The cleanup flag stops a slow old response from overwriting a newer one.

Check yourself

1. When do you use useMemo?
   Answer: After measuring that a calculation is slow.
2. What goes in the dependency list?
   Answer: Every value from the component the effect uses.

Lesson 7.2: SSR vs SSG in Next.js

What you learn

- **SSG.** Pages are built once at build time. Fast and cheap; content can be refreshed on a schedule.
- **SSR.** Pages are built on each request. Use for personalised or frequently changing data.
- **Choosing.** Marketing pages and docs suit SSG; dashboards and account pages suit SSR.

Tools and resources (official sites, not checked live)

- Next.js rendering

ASSIGNMENT

Practice for lesson 7.2

Business problem: Pick a strategy for five pages.

Steps

1. List five pages.
2. Choose SSG or SSR for each.
3. Give one reason each.

Expected output: A table with reasons.

SOLUTION (ANSWER KEY)

Home, blog and pricing: SSG. Account and live order status: SSR.

Check yourself

1. Which is faster to serve?
   Answer: SSG.
2. When must you use SSR?
   Answer: When content depends on the request or changes constantly.

8. E-Commerce Developer

Lesson 8.1: Cart logic, checkout, PCI-DSS and multi-currency

What you learn

- **Cart logic.** Total = items, discounts, shipping and tax in a fixed order. Store money as integers (paise or cents) to avoid rounding errors.
- **Checkout.** Validate stock and address, then create an order before payment is confirmed; confirm only after the payment provider says it succeeded.
- **PCI-DSS.** The standard for protecting card data. Using a hosted payment page or provider fields keeps card numbers off your servers.
- **Multi-currency.** Show prices in the buyer's currency with a rate you can explain, round consistently and charge in the currency shown.

Tools and resources (official sites, not checked live)

- PCI Security Standards Council

ASSIGNMENT

Practice for lesson 8.1

Business problem: Compute a cart total.

Steps

1. Write a function with items, a coupon and shipping.
2. Use integer money.
3. Test two coupon cases.

Expected output: Totals that match a hand calculation.

SOLUTION (ANSWER KEY)

The earlier Development and IT textbook has a tested cart function you can reuse.

Check yourself

1. Why integer money?
   Answer: Floating point rounding creates cent errors.
2. How do you stay out of PCI scope?
   Answer: Use hosted fields or pages so card numbers never touch your server.

Lesson 8.2: Payment gateway integration

What you learn

- **Stripe, Razorpay and PayPal.** Each provides checkout pages, SDKs and webhooks. Razorpay is widely used in India; Stripe and PayPal serve many countries. Check availability and fees for your country on each site.
- **Flow.** Create an order on your server, open the provider's checkout, receive a webhook, verify its signature, then mark the order paid.
- **Test mode.** Use the provider's test keys and test cards; never test with real cards.

Tools and resources (official sites, not checked live)

- Stripe docs
- Razorpay docs
- PayPal developer

ASSIGNMENT

Practice for lesson 8.2

Business problem: Add test-mode checkout.

Steps

1. Create a test account.
2. Create an order on the server.
3. Handle the webhook and verify the signature.

Expected output: A test payment that marks an order paid exactly once.

SOLUTION (ANSWER KEY)

Store the provider's payment ID on the order and ignore a webhook you have already processed.

Check yourself

1. Why verify the webhook?
   Answer: To stop anyone faking a paid order.
2. Why use idempotent handling?
   Answer: Providers can send the same event twice.

9. React.js and Next.js Advanced Developer

Lesson 9.1: Next.js App Router, server actions, edge functions and middleware

What you learn

- **App Router.** Folders define routes; layouts nest; server components run on the server by default.
- **Server actions.** Functions that run on the server and are called from forms or components. Validate input and check the user inside them.

- **Edge functions and middleware.** Run close to the user before a request completes: redirects, auth checks, headers. Keep them small.
- **Security.** Treat every server action as a public endpoint.

Tools and resources (official sites, not checked live)

- Next.js App Router

ASSIGNMENT

Practice for lesson 9.1

Business problem: Protect a route.

Steps

1. Add middleware that redirects signed-out users from /account.
2. Add a server action with validation.
3. Test both.

Expected output: Signed-out requests redirect; invalid input is rejected.

SOLUTION (ANSWER KEY)

Middleware checks a session cookie and redirects. The action validates input again because it is a public endpoint.

Check yourself

1. Are server actions private?
   Answer: No; validate and authorise each call.
2. What is middleware for?
   Answer: Work that must happen before the page, such as redirects.

10. Bubble.io Developer

Lesson 10.1: Visual database modelling, privacy rules, API Connector and plugins

What you learn

- **Data types.** Model things (User, Order) with fields and relationships.
- **Privacy rules.** Control who can find and see each type. Set them first; the default can expose data.
- **API Connector.** Call outside APIs with set headers and parameters; keep keys private.
- **Plugins.** Use trusted plugins and consider writing your own for repeated needs.

Tools and resources (official sites, not checked live)

- Bubble manual

ASSIGNMENT

Practice for lesson 10.1

Business problem: Lock down a data type.

Steps

1. Create an Order type.
2. Add a rule: only the owner can find and view.
3. Test as another user.

Expected output: Another user sees nothing.

SOLUTION (ANSWER KEY)

Privacy rules apply to searches and direct access. Always test with a second account.

Check yourself

1. Why set privacy rules early?
   Answer: Data can be exposed by default.
2. Where do API keys go?
   Answer: In private fields of the API Connector.

DEPARTMENT 2

AI and Automation

Lessons for Department 2. Each role has numbered lessons with what you learn, official tools, a practice task with an answer, and short questions. The matching skill textbook is the Master Freelancing AI and Automation book.

1. AI Video Creator and Editor

Makes short videos with AI image and video tools, then finishes them with sound and subtitles.

Lesson 1.1: Image-to-video and text-to-video platforms

What you learn

- **Text-to-video vs image-to-video.** Text-to-video builds a clip from a prompt. Image-to-video animates a picture you chose, which gives more control over how the first frame looks.
- **Runway ML.** A full creative suite with video generation, editing tools and camera controls.
- **Kling AI.** A video generator known for strong motion and realism; it uses a credit system.
- **Pika Labs.** Short clips and effects; quick for stylised results.
- **HeyGen.** Talking avatars and translated video with lip-sync; useful for explainers and training.
- **Before you start.** Plans, credits and features change often. Read each tool's current pricing and its rules on commercial use, likeness and consent before selling any work.

Tools and resources (official sites, not checked live)

- Runway
- Kling AI
- Pika
- HeyGen

ASSIGNMENT

Practice for lesson 1.1

Business problem: Compare four tools on one idea.

Steps

1. Pick one 5-second scene.
2. Generate it in two tools, using the same words.
3. Score each for motion, faces, text and cost in credits.
4. Write a one-paragraph recommendation.

Expected output: A comparison table and a recommendation.

SOLUTION (ANSWER KEY)

Use a 1 to 5 score for each item and record the credits used. The best tool depends on the scene, so keep the table for next time.

Check yourself

1. Why start from an image?
   Answer: You control the first frame.
2. What should you check before selling AI video?
   Answer: The tool's commercial-use terms and any likeness or consent limits.

Lesson 1.2: Prompt engineering for consistent scenes and characters

What you learn

- **The consistency formula.** [Subject description] + [Style and lighting] + [Camera lens specs] + [Seed number or reference image]. Reuse the same subject words in every shot.
- **Reference images.** Most tools let you upload a reference of the character. This is more reliable than text alone.
- **Seeds.** Some tools expose a seed number; reusing it with similar prompts gives similar results. Not every tool supports it.
- **Keep a character sheet.** Write one fixed paragraph describing the character and paste it into every prompt.
- **Aspect ratio.** The flag --ar 16:9 is Midjourney syntax. In Runway, Kling and Pika you set the aspect ratio in the tool's own settings.

Tools and resources (official sites, not checked live)

- Runway learn
- Midjourney docs

ASSIGNMENT

Practice for lesson 1.2

Business problem: Write a character sheet and three shots.

Steps

1. Write a 40-word description of one character.
2. Write three shot prompts that each start with that description.
3. Generate them and compare faces and clothes.

Expected output: Three shots where the character is recognisable.

SOLUTION (ANSWER KEY)

Keep the description word for word. If a detail drifts, add it to the sheet. Use the same reference image in every shot.

Check yourself

1. Why reuse the same words?
   Answer: Small wording changes cause the character to drift.
2. Where do you set aspect ratio outside Midjourney?
   Answer: In the tool's settings.

Lesson 1.3: Video production case studies and prompts

What you learn

- **How to use these.** Treat each prompt as a starting point. Change the subject, then adjust the camera and light. Remove the --ar flag outside Midjourney and set the ratio in the tool.
- **Respect.** The mythological prompt depicts a religious figure. Keep the depiction respectful and check the tool's content rules and the audience before publishing.
- **Case study A: cartoon animation.** 3D Pixar-style animation of a brave young explorer holding a glowing map inside an ancient temple, soft cinematic lighting, 8k resolution, highly detailed character design, 24fps --ar 16:9
- **Case study B: epic mythological scene (Ramayan theme).** Cinematic shot of Lord Rama standing on an ocean cliff holding a glowing bow, dramatic stormy sky, golden armour detailing, photorealistic render, wide-angle lens, Unreal Engine 5 render style --ar 16:9
- **Case study C: dynamic action scene.** Action camera tracking shot of a martial arts duel on a rainy neon-lit roof at night, slow-motion impact shot, wet reflections, cinematic motion blur, 80mm lens, anamorphic lighting --ar 16:9
- **Case study D: multi-character narrative.** Wide cinematic shot in a grand stone throne room: a regal King in velvet robes sitting beside a graceful Queen, while a shadowed figure in a dark hooded cloak approaches from the marble aisle, dramatic chiaroscuro lighting, movie still --ar 16:9

Tools and resources (official sites, not checked live)

- Runway
- Kling AI

ASSIGNMENT

Practice for lesson 1.3

Business problem: Produce one of the four scenes.

Steps

1. Choose a case study.
2. Generate a still, then animate it.
3. Change one element (time of day) and generate again.
4. Write what changed.

Expected output: Two versions of one scene and a short note.

SOLUTION (ANSWER KEY)

Changing only one element shows you what the prompt controls. Keep the winning prompt in your library.

Check yourself

1. Why change only one element?
   Answer: So you can see its effect.

2. Which parts of case study D keep the multi-character scene clear?
   Answer: Positions (beside, from the aisle) and lighting that separates the figures.

Lesson 1.4: Credit optimisation and efficiency

What you learn

- **Two-stage method.** Create the base image in an image tool (Midjourney or Leonardo.ai), approve it, then animate only the approved image in Runway or Kling. Images cost less than video, so you iterate cheaply.
- **Short test clips.** Generate 3 to 5 second tests before a longer clip.
- **Reuse.** Animate several shots from one approved set of images.
- **Track.** Keep a sheet of credits spent per scene.

Tools and resources (official sites, not checked live)

- Leonardo.ai
- Midjourney

ASSIGNMENT

Practice for lesson 1.4

Business problem: Plan a 30-second ad on a credit budget.

Steps

1. Break it into six 5-second shots.
2. Plan images first.
3. Estimate credits using each tool's current price list.
4. Leave 20 percent for re-runs.

Expected output: A shot list with a credit budget.

SOLUTION (ANSWER KEY)

Check each tool's price page for current credit costs, then multiply by the shots and the re-runs.

Check yourself

1. Why iterate on images first?
   Answer: They cost less than video.
2. Why keep a 20 percent reserve?
   Answer: Some generations fail or need a redo.

Lesson 1.5: Post-production and audio sync

What you learn

- **Voice-over.** ElevenLabs creates voices from text. Use voices you are licensed to use and never clone a person's voice without their written consent.
- **Lip-sync.** Match mouth movement to the audio in the video tool's lip-sync feature or in HeyGen.

- **Subtitles.** CapCut can auto-generate subtitles; always proofread them.
- **Score and mix.** Descript edits audio like text and helps arrange music; keep voice louder than music.

Tools and resources (official sites, not checked live)

- ElevenLabs
- CapCut
- Descript

ASSIGNMENT

Practice for lesson 1.5

Business problem: Finish a 30-second clip.

Steps

1. Record or generate a voice-over.
2. Sync it.
3. Add and proofread subtitles.
4. Mix music under the voice.

Expected output: A finished clip with captions and balanced sound.

SOLUTION (ANSWER KEY)

Play it with the sound off to check the subtitles, then with sound only to check the mix.

Check yourself

1. Why proofread auto-subtitles?
   Answer: They contain errors, especially with names.
2. What consent is needed to clone a voice?
   Answer: The speaker's clear written consent.

2. AI Integration Developer

Connects language models and search to real applications.

Lesson 2.1: OpenAI API and Anthropic SDK integration in Node.js and Python

What you learn

- **The call.** Send messages (a system message and the user's message) to the model API and read the reply. Both providers have official SDKs for Node.js and Python.
- **API keys.** Keep keys in environment variables on the server, never in front-end code or the repository.
- **Handling errors.** Expect rate limits and timeouts; retry with a short wait and give the user a clear message.
- **Costs.** You pay for input and output tokens; set limits and log usage.
- **Model names.** Names change; read the provider's current model list rather than copying an old name.

Tools and resources (official sites, not checked live)

- Anthropic documentation
- OpenAI documentation

ASSIGNMENT

Practice for lesson 2.1

Business problem: Build a one-route summariser.

Steps

1. Create a server route that accepts text.
2. Call the model with a system instruction.
3. Return the summary and token usage.
4. Add a length limit.

Expected output: A route that returns a short summary and refuses huge inputs.

SOLUTION (ANSWER KEY)

Reject input above your limit before calling the model to control cost.

Check yourself

1. Where does the API key live?
   Answer: In a server environment variable.
2. Why a length limit?
   Answer: To control cost and avoid failures.

Lesson 2.2: Vector databases: Pinecone, ChromaDB and Weaviate

What you learn

- **Embeddings.** An embedding turns text into a list of numbers so similar meanings are near each other.
- **Similarity search.** Find the stored items closest to the question's embedding, often by cosine similarity (the code below shows the idea with tiny vectors).
- **Choosing.** ChromaDB is simple to run locally. Pinecone is a managed service. Weaviate is open source and can be self-hosted or managed.

Tools and resources (official sites, not checked live)

- Pinecone
- Chroma
- Weaviate

ASSIGNMENT

Practice for lesson 2.2

Business problem: Run a similarity search.

Steps

1. Run the tested toy search below.
2. Change the query numbers.
3. See which item wins.

Expected output: A top match for each query.

SOLUTION (ANSWER KEY)

PYTHON

```
import math
def cosine(a, b):
    dot = sum(x*y for x, y in zip(a, b)); na = math.sqrt(sum(x*x for x in a)); nb =
math.sqrt(sum(y*y for y in b))
    return dot/(na*nb) if na and nb else 0.0
def top_k(query_vec, items, k=1):
    scored = sorted(((cosine(query_vec, v), name) for name, v in items.items()), reverse=True)
    return [(name, round(s, 3)) for s, name in scored[:k]]
# toy 3-number "embeddings": [about money, about time, about place]
items = {"refund policy": [0.9, 0.3, 0.0], "class timings": [0.1, 0.95, 0.1], "campus
address": [0.0, 0.1, 0.95]}
```

Real embeddings have hundreds or thousands of numbers, but the ranking works the same way.

Check yourself

1. What does an embedding capture?
   Answer: Meaning, as numbers.
2. What does cosine similarity measure?
   Answer: How closely two vectors point the same way.

Lesson 2.3: Retrieval-augmented generation (RAG) and document indexing

What you learn

- **The pipeline.** Split documents into chunks, embed and index them, retrieve the best chunks for a question, give them to the model with the question, and ask it to answer only from them and cite the source.
- **Chunking.** Chunks of a few hundred words with a little overlap keep ideas together.
- **Failure modes.** Wrong chunk retrieved, answer not in the documents, or the model ignoring the context. Test with real questions and add 'say you do not know' to the instruction.
- **Privacy.** Index only documents you may use and respect who can see what.

Tools and resources (official sites, not checked live)

- Anthropic RAG guidance
- LlamaIndex

ASSIGNMENT

Practice for lesson 2.3

Business problem: Test a mini RAG.

Steps

1. Run the tested code.
2. Ask a question that is not in the documents.
3. Confirm it says it was not found.

Expected output: Answers cite a source, and unknown questions are refused.

SOLUTION (ANSWER KEY)

PYTHON

```
import re
from collections import Counter

STOP = {"the", "and", "for", "are", "you", "your", "how", "what", "does", "long", "take",
        "have", "with", "from", "that", "this", "after", "within", "each", "shown", "before",
"run", "our", "can", "will"}
DOCS = {
    "refund": "Refunds are processed within 7 working days after a written request.",
    "fees": "Course fees are shown on each program page and confirmed in writing before
payment.",
    "timings": "Live classes run on Saturdays and Sundays in the evening.",
}

def tokens(text):
    return [w for w in re.findall(r"[a-z]+", text.lower()) if len(w) > 2 and w not in STOP]

def retrieve(question, docs=DOCS, k=1):
    q = Counter(tokens(question))
    scored = []
    for key, text in docs.items():
        d = Counter(tokens(text))
        score = sum(min(q[w], d[w]) for w in q)
        scored.append((score, key, text))
    scored.sort(reverse=True)
    return [(key, text) for score, key, text in scored[:k] if score > 0]

def answer(question):
    hits = retrieve(question)
    if not hits:
        return "Not found in our documents. A team member will reply."
    key, text = hits[0]
    return f"{text} [source: {key}]"
```

The answer includes a source tag. The pizza question returns 'Not found', which is the safe behaviour.

Check yourself

1. Why cite the source?
   Answer: So users can check the answer.

2. What should the bot do when the answer is not in the documents?
   Answer: Say so and hand over to a person.

3. Chatbot Developer

Lesson 3.1: RAG-powered bots in Voiceflow, Botpress and Flowise

What you learn

- **Voiceflow.** Visual conversation design with a knowledge base.
- **Botpress.** A builder with knowledge bases, flows and channel connections.
- **Flowise.** An open-source drag-and-drop builder for language-model flows, which can be self-hosted.
- **Design rules.** Greet, state what the bot can do, answer from the knowledge base, offer a human, log unanswered questions and review them weekly.

Tools and resources (official sites, not checked live)

- Voiceflow
- Botpress
- Flowise

ASSIGNMENT

Practice for lesson 3.1

Business problem: Build a support bot for ten FAQs.

Steps

1. Load ten FAQs.
2. Add a fallback to a human.
3. Test 20 questions, five of them off-topic.

Expected output: A bot that answers 80 percent correctly and hands over the rest.

SOLUTION (ANSWER KEY)

Record each test question and the result. The off-topic questions should trigger the fallback.

Check yourself

1. Why add a human handover?
   Answer: The bot cannot solve everything.
2. What do you review weekly?
   Answer: Unanswered and wrongly answered questions.

Lesson 3.2: WhatsApp Business API integration via Twilio and Wati

What you learn

- **Consent.** Message only people who opted in, and follow WhatsApp's business and commerce policies.

- **Templates.** Business-started conversations use pre-approved templates; replies within the service window can be free-form. Rules and prices change, so read current documents.
- **Setup.** Get a business account and phone number through the provider, set a webhook for incoming messages, and send replies through the API.
- **Providers.** Twilio gives developer APIs; Wati is a no-code platform.

Tools and resources (official sites, not checked live)

- Twilio WhatsApp
- Wati
- WhatsApp Business Platform

ASSIGNMENT

Practice for lesson 3.2

Business problem: Plan a WhatsApp order-status bot.

Steps

1. List the messages the bot will send.
2. Mark which need templates.
3. Write the opt-in text.

Expected output: A message map with opt-in and template notes.

SOLUTION (ANSWER KEY)

Order updates sent first by the business usually need a template. Keep an opt-in record for each customer.

Check yourself

1. Do you need opt-in?
   Answer: Yes.
2. When are templates needed?
   Answer: For messages the business starts, outside the allowed reply window.

4. Machine Learning and AI Engineer

Lesson 4.1: Data preprocessing and model training (Pandas, NumPy, Scikit-Learn, PyTorch)

What you learn

- **Preprocessing.** Clean missing values, fix types, encode categories and scale numbers where the model needs it.
- **Train and test split.** Train on one part of the data and measure on rows the model has not seen.
- **Scikit-Learn.** Classical models with a common fit and predict pattern.
- **PyTorch.** Flexible framework for neural networks; use it when classical models are not enough.

- **Honest evaluation.** One accuracy number on a tiny sample proves little. Use more data and the right metric for the business.

Tools and resources (official sites, not checked live)

- Pandas
- NumPy
- Scikit-learn
- PyTorch

ASSIGNMENT

Practice for lesson 4.1

Business problem: Train a purchase predictor.

Steps

1. Run the tested code.
2. Change test_size.
3. Explain why the score is not proof with only 20 rows.

Expected output: Accuracy on held-out rows, and your note on its limits.

SOLUTION (ANSWER KEY)

PYTHON

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
df = pd.DataFrame({"visits":[1,2,8,9,3,10,2,7,1,9,8,2,6,7,3,1,9,10,2,8],
                   "bought":[0,0,1,1,0,1,0,1,0,1,1,0,1,1,0,0,1,1,0,1]})
df = df.dropna()                                    # preprocessing: remove missing rows
X_train, X_test, y_train, y_test = train_test_split(df[["visits"]], df["bought"],
test_size=0.25, random_state=0)
model = LogisticRegression().fit(X_train, y_train)
acc = accuracy_score(y_test, model.predict(X_test))
print("accuracy on held-out rows:", acc)
assert acc >= 0.8
print("ml test passed")
```

The tiny sample makes the high score unreliable. Real projects need far more rows and cross-validation.

Check yourself

1. Why hold out test rows?
   Answer: To measure on data the model has not seen.
2. Why is 20 rows too few?
   Answer: The result is unstable and may not generalise.

5. Automation Expert (Make and Zapier)

Lesson 5.1: Multi-step scenarios, JSON parsing, array aggregators and webhooks

What you learn

- **Routing.** Use routers or paths so different conditions follow different steps, with a fallback path.
- **JSON parsing.** Convert incoming text into fields you can map; check the structure first.
- **Array aggregator.** Collects many items into one bundle or total, for example orders per city (shown in code below).
- **Webhook listeners.** A URL that starts the scenario when another app sends data. Verify the sender and handle repeats.

Tools and resources (official sites, not checked live)

- Make help
- Zapier help

ASSIGNMENT

Practice for lesson 5.1

Business problem: Total orders by city.

Steps

1. Run the tested aggregator.
2. Add a new city to the data.
3. Check the totals.

Expected output: Totals per city.

SOLUTION (ANSWER KEY)

PYTHON

```
import json
raw = '{"orders":[{"id":1,"city":"Pune","amount":500},{"id":2,"city":"Pune","amount":300},
{"id":3,"city":"Delhi","amount":700}]}'
def aggregate_by(key, rows, value):
    out = {}
    for r in rows:
        out[r[key]] = out.get(r[key], 0) + r[value]
    return out
```

Make's aggregators do the same job without code: they group items and sum a field.

Check yourself

1. Why verify webhook senders?
   Answer: To block fake requests.
2. What does an aggregator do?
   Answer: Combines many items into one result.

6. n8n Expert

Lesson 6.1: Self-hosting n8n with Docker on DigitalOcean or AWS

What you learn

- **Why self-host.** More control and no per-run limits on the open-source edition, in exchange for maintaining the server. Check n8n's current licence terms for commercial use.
- **Steps.** Create a small server, install Docker, run the n8n image with a persistent volume, put it behind HTTPS with a reverse proxy, set a strong encryption key and sign-in, and back up the volume.
- **Security.** Do not expose the editor without authentication; keep the server updated.

Tools and resources (official sites, not checked live)

- n8n self-hosting
- DigitalOcean docs
- AWS docs

ASSIGNMENT

Practice for lesson 6.1

Business problem: Write a deployment checklist.

Steps

1. List the server size and cost from current price pages.
2. List the Docker, HTTPS, backup and update steps.
3. Add a restore test.

Expected output: A checklist another person can follow.

SOLUTION (ANSWER KEY)

Include where the encryption key is stored and how you restore from a backup.

Check yourself

1. What must persist across restarts?
   Answer: The data volume.
2. Why HTTPS?
   Answer: To protect sign-ins and webhooks.

Lesson 6.2: Custom JavaScript nodes, error workflows and API polling

What you learn

- **Code node.** Run JavaScript on items to reshape data.
- **Error workflows.** A separate workflow that runs when another fails and alerts you with the workflow name and the error.
- **Polling.** Call an API on a schedule; remember the last item processed to avoid duplicates, and respect rate limits.

Tools and resources (official sites, not checked live)

- n8n Code node

ASSIGNMENT

Practice for lesson 6.2

Business problem: Add an error alert.

Steps

1. Create an error workflow.
2. Link it to a workflow.
3. Force a failure.
4. Confirm the alert.

Expected output: An alert naming the failed workflow.

SOLUTION (ANSWER KEY)

Test failures on purpose; an untested alert may never fire when you need it.

Check yourself

1. Why store the last processed ID?
   Answer: To avoid handling the same item twice.
2. Why test the alert?
   Answer: To be sure it works.

7. Vibe Coder

Lesson 7.1: AI-first coding workflows with Cursor, Replit Agent and Claude

What you learn

- **Workflow.** Describe the feature, let the assistant plan, build in small steps, run and read the output, then review the diff before accepting.
- **Prompt structure.** Role, task, constraints, files involved and how to test. Short, specific prompts beat long vague ones.
- **Tools.** Cursor edits your local project; Replit Agent builds and hosts in the browser; Claude can write and review code in chat or in an editor.
- **Model names.** Model versions change quickly. Use the best current model the tool offers rather than a named old one.
- **Review.** Read generated code, run tests, and watch for secrets and insecure defaults.

Tools and resources (official sites, not checked live)

- Cursor
- Replit
- Anthropic docs

ASSIGNMENT

Practice for lesson 7.1

Business problem: Add a feature in small steps.

Steps

1. Ask for a plan.
2. Implement step 1 and run it.
3. Ask for tests.
4. Review every change.

Expected output: A working feature with tests and a reviewed diff.

SOLUTION (ANSWER KEY)

Small steps make mistakes easy to see and undo.

Check yourself

1. Why work in small steps?
   Answer: Errors are easier to find.
2. What must you always do with generated code?
   Answer: Read, run and test it.

8. Claude and LLM Specialist

Lesson 8.1: System prompt optimisation, context window management and artifacts

What you learn

- **System prompts.** State the role, the audience, rules, format and what to do when unsure. Test with hard cases and keep versions.
- **Context window.** A model can read only so much text at once. Keep instructions, the relevant documents and recent turns; trim or summarise the rest (see the tested code).
- **Artifacts.** In Claude, artifacts are separate documents, pages or small apps produced beside the chat, which you can edit and share.
- **Evaluation.** Write 20 test questions and compare prompt versions on them.

Tools and resources (official sites, not checked live)

- Anthropic prompt engineering
- Claude

ASSIGNMENT

Practice for lesson 8.1

Business problem: Trim a long chat.

Steps

1. Run the tested trimmer.
2. Change the budget.
3. See which messages remain.

Expected output: The system message and newest messages that fit.

SOLUTION (ANSWER KEY)

PYTHON

```
def rough_tokens(text): return max(1, len(text) // 4)   # rule of thumb: about 4 characters
per token
def trim_history(messages, budget):
    """Keep the system message and the newest messages that fit inside the token budget."""
    system, rest = messages[0], messages[1:]
    kept, used = [], rough_tokens(system["content"])
    for m in reversed(rest):
        t = rough_tokens(m["content"])
        if used + t > budget: break
        kept.append(m); used += t
    return [system] + list(reversed(kept))
```

Real tools count tokens exactly. The 4-characters rule is only an estimate.

Check yourself

1. What should always stay in context?
   Answer: The system instructions.
2. How do you compare two system prompts?
   Answer: Run both on the same test questions.

9. AI Consultant

Lesson 9.1: Corporate AI audits, ROI models and technology stack mapping

What you learn

- **Audit.** List the processes, who does them, hours spent, data involved and risks. Rank by value and ease.
- **ROI.** Annual saving = hours saved x hourly cost x weeks. Subtract build cost and yearly tools. Payback = build cost / monthly net saving.
- **Stack mapping.** Draw the tools, data and owners. Mark duplicates and gaps.
- **Honesty.** Label estimates as estimates and list assumptions.

Tools and resources (official sites, not checked live)

- NIST AI Risk Management Framework

ASSIGNMENT

Practice for lesson 9.1

Business problem: Calculate payback for one automation.

Steps

1. Run the tested ROI function.
2. Change the hours saved.
3. Explain the result to a manager.

Expected output: Payback in months and first-year ROI.

SOLUTION (ANSWER KEY)

PYTHON

```
def automation_roi(hours_saved_per_week, hourly_cost, weeks_per_year, build_cost,
monthly_tool_cost):
    annual_saving = hours_saved_per_week * hourly_cost * weeks_per_year
    annual_tool = monthly_tool_cost * 12
    first_year_net = annual_saving - build_cost - annual_tool
    net_per_year_after_build = annual_saving - annual_tool
    payback_months = build_cost / (net_per_year_after_build / 12) if net_per_year_after_build
> 0 else float("inf")
    roi_percent = first_year_net / (build_cost + annual_tool) * 100
    return {"annual_saving": annual_saving, "annual_tool": annual_tool,
            "first_year_net": first_year_net, "payback_months": round(payback_months, 1),
            "roi_percent": round(roi_percent, 1)}
```

With these example figures payback is 10 months and first-year ROI is 14.3 percent. All inputs are illustrative.

Check yourself

1. What is payback?
   Answer: Months to earn back the build cost.
2. Why list assumptions?
   Answer: So the client can challenge the numbers.

DEPARTMENT 3

Design and Creative

Lessons for Department 3. Each role has numbered lessons with what you learn, official tools, a practice task with an answer, and short questions. The matching skill textbook is the Master Freelancing Design and Creative book.

1. Logo Designer

Designs marks that work at every size and fit the industry.

Lesson 1.1: Industry-specific logo systems

What you learn

- **E-commerce logos.** Clean typography, a simple vector icon and high readability at small sizes such as a browser tab, an app icon and a packaging label.
- **Fintech logos.** Geometric symmetry, calm trust-building colours (deep blues and greens are common) and abstract marks suggesting growth, safety or flow.
- **System, not one file.** Deliver a primary logo, a compact mark, one-colour and reversed versions, clear space and minimum size.
- **Originality.** Check that the idea is not a copy of an existing mark; search before you deliver.

Tools and resources (official sites, not checked live)

- Brand New

ASSIGNMENT

Practice for lesson 1.1

Business problem: Design two marks for one fintech.

Steps

1. Write the brand idea in one line.
2. Sketch six ideas and pick two.
3. Test each at 16 px and in one colour.

Expected output: Two readable marks and a note on which is stronger.

SOLUTION (ANSWER KEY)

A mark that survives 16 px in a single colour is robust. Record why you chose it.

Check yourself

1. What must an e-commerce logo do at small sizes?
   Answer: Stay readable.
2. Why deliver one-colour versions?
   Answer: For print, stamps and low-colour uses.

Lesson 1.2: Tooling and workflow

What you learn

- **Vector tools.** Adobe Illustrator and the free Inkscape draw shapes that scale without loss.
- **Font pairing.** Pick one display and one text font from Google Fonts or Adobe Fonts; check the licence for commercial use.
- **Animation.** After Effects can animate a logo reveal; keep it under five seconds.

Tools and resources (official sites, not checked live)

- Inkscape
- Google Fonts
- Adobe Fonts

ASSIGNMENT

Practice for lesson 1.2

Business problem: Build a logo in vectors.

Steps

1. Draw it with as few points as you can.
2. Pair two fonts.
3. Export SVG and PNG.

Expected output: A vector file and a font note with licences.

SOLUTION (ANSWER KEY)

Keep the file organised with named layers so the client can edit it.

Check yourself

1. Why vectors?
   Answer: They scale without losing quality.
2. What do you check about fonts?
   Answer: The licence.

Lesson 1.3: Asset sourcing and typography

What you learn

- **Inspiration, not copying.** Behance, Dribbble and Brand New show current work and case studies. Learn principles; do not reproduce another designer's work.
- **Fontsource.** Open-source fonts packaged for web projects; check each font's licence.
- **Mood boards.** Collect references, then write what you will take from them (spacing, colour, mood).

Tools and resources (official sites, not checked live)

- Behance
- Dribbble

- Fontsource

ASSIGNMENT

Practice for lesson 1.3

Business problem: Create a mood board.

Steps

1. Collect 12 references.
2. Write three adjectives.
3. State what you take from each.

Expected output: A board with reasons.

SOLUTION (ANSWER KEY)

The reasons show process and keep you from copying.

Check yourself

1. Why write reasons beside references?
   Answer: To learn principles and avoid copying.
2. What about font licences?
   Answer: Check them before commercial use.

2. Graphic Designer

Lesson 2.1: Principles of layout

What you learn

- **Visual hierarchy.** Size, weight, colour and position tell the eye where to look first.
- **Negative space.** Empty space gives focus and calm.
- **Colour theory.** Use a limited palette with one accent. RGB is for screens; CMYK is for print, so convert and proof before printing.
- **Contrast.** Text must be readable. The checker below measures the WCAG contrast ratio.

Tools and resources (official sites, not checked live)

- Adobe colour

ASSIGNMENT

Practice for lesson 2.1

Business problem: Check a palette.

Steps

1. Run the tested contrast checker.
2. Test your text and background.
3. Fix any failing pair.

Expected output: All text pairs at 4.5:1 or better.

SOLUTION (ANSWER KEY)

PYTHON

```
def _lin(c):
    c = c / 255
    return c / 12.92 if c <= 0.03928 else ((c + 0.055) / 1.055) ** 2.4

def luminance(hex_colour):
    h = hex_colour.lstrip("#")
    r, g, b = (int(h[i:i + 2], 16) for i in (0, 2, 4))
    return 0.2126 * _lin(r) + 0.7152 * _lin(g) + 0.0722 * _lin(b)

def contrast_ratio(fg, bg):
    l1, l2 = sorted([luminance(fg), luminance(bg)], reverse=True)
    return round((l1 + 0.05) / (l2 + 0.05), 2)

def passes_aa(fg, bg, large_text=False):
    return contrast_ratio(fg, bg) >= (3.0 if large_text else 4.5)
```

Dark ink on white scores 16.92. A light grey on white fails the 4.5 test.

Check yourself

1. RGB or CMYK for print?
   Answer: CMYK.
2. What ratio does normal text need?
   Answer: 4.5:1.

Lesson 2.2: Social media, ads and print collateral

What you learn

- **Canva, Figma and Photoshop.** Canva is fast for templates, Figma for shared components, Photoshop for photo work.
- **Sizes.** Use each platform's current recommended sizes and safe areas.
- **Print.** 300 dpi, bleed and CMYK; ask the printer for specs.

Tools and resources (official sites, not checked live)

- Canva
- Figma

- Photoshop

ASSIGNMENT

Practice for lesson 2.2

Business problem: Make a campaign set.

Steps

1. Design one post, one display ad and one flyer from a single palette.
2. Export in the right sizes.
3. Proof the print version.

Expected output: Three consistent pieces.

SOLUTION (ANSWER KEY)

One grid, two fonts and one accent colour hold the set together.

Check yourself

1. What is bleed?
   Answer: Extra image beyond the trim so no white edge shows.
2. Why a shared palette?
   Answer: Consistency.

3. Web Designer (UI focus)

Lesson 3.1: Design systems in Figma

What you learn

- **Components.** Reusable elements with variants (primary, secondary, disabled).
- **Auto layout.** Frames that resize with their content; check Figma's current documentation for its features.
- **Variables and design tokens.** Named values for colour, spacing and type, with modes such as light and dark.
- **Handoff.** Name layers clearly and document states.

Tools and resources (official sites, not checked live)

- Figma help

ASSIGNMENT

Practice for lesson 3.1

Business problem: Build a button component.

Steps

1. Create variants for primary, secondary and disabled.
2. Bind colours to variables.
3. Switch to a dark mode.

Expected output: A button that changes by variant and mode.

SOLUTION (ANSWER KEY)

Tokens let one change update every use.

Check yourself

1. What is a design token?
   Answer: A named design value.
2. Why use variants?
   Answer: One component, many states.

4. Video Editor

Lesson 4.1: Timeline editing, J/L cuts, sound design and colour grading

What you learn

- **Timeline.** Cut to the beat of the story; remove every frame that does not help.
- **J and L cuts.** The audio from the next shot starts before the picture (J) or the audio of the last shot continues over the next picture (L); they smooth transitions.
- **Sound design.** Dialogue first, then effects, then music; clean noise.
- **Colour grading with LUTs.** A LUT applies a colour look. Balance exposure and white balance first, then apply the look.

Tools and resources (official sites, not checked live)

- DaVinci Resolve
- Premiere Pro

ASSIGNMENT

Practice for lesson 4.1

Business problem: Edit a 60-second interview.

Steps

1. Cut to the best answers.
2. Add one J cut and one L cut.
3. Grade with a LUT.

Expected output: A smooth 60-second clip.

SOLUTION (ANSWER KEY)

Play it without looking to check that the audio flows.

Check yourself

1. What is an L cut?
   Answer: Audio from the previous shot continues over the next picture.
2. What do you fix before a LUT?
   Answer: Exposure and white balance.

5. Photo Editor and Photoshop Expert

Lesson 5.1: Non-destructive editing, frequency separation and compositing

What you learn

- **Non-destructive.** Use layers, adjustment layers, masks and smart objects so edits can be changed.
- **Frequency separation.** Splits colour and tone from fine texture so skin can be smoothed while keeping natural texture. Keep retouching subtle and honest.
- **Compositing.** Match light direction, colour and perspective.
- **Colour correction.** Set a neutral white point first.

Tools and resources (official sites, not checked live)

- Photoshop tutorials

ASSIGNMENT

Practice for lesson 5.1

Business problem: Retouch a portrait.

Steps

1. Work on layers only.
2. Remove blemishes with frequency separation.
3. Compare before and after.

Expected output: A natural result with all edits on separate layers.

SOLUTION (ANSWER KEY)

If you can hide a layer and the original returns, the edit is non-destructive.

Check yourself

1. Why use adjustment layers?
   Answer: They can be changed later.
2. Why keep retouching subtle?
   Answer: It looks natural and honest.

6. UX Designer

Lesson 6.1: User research, wireframing, information architecture and usability testing

What you learn

- **Research.** Interview real users to learn goals and problems.
- **Wireframes.** Low-fidelity layouts that test structure before visual design.
- **Information architecture.** How content is grouped and labelled; test with card sorting.
- **Usability testing.** Watch five people try tasks and note where they struggle; Maze and Figma prototypes support remote tests.

Tools and resources (official sites, not checked live)

- Maze
- Nielsen Norman Group

ASSIGNMENT

Practice for lesson 6.1

Business problem: Test a prototype.

Steps

1. Write three tasks.
2. Test with five people.
3. List the problems by severity.

Expected output: A prioritised problem list.

SOLUTION (ANSWER KEY)

If three of five fail the same step, fix it first.

Check yourself

1. Why test with real users?
   Answer: Designers cannot see their own blind spots.
2. What does information architecture decide?
   Answer: Grouping and labels.

7. Presentation Designer

Lesson 7.1: Slide master, data visualisation and pitch decks

What you learn

- **Slide master.** Set fonts, colours and layouts once so every slide matches.
- **Chart clean-up.** Remove clutter, label directly, use one highlight colour and a title that states the finding.
- **Pitch deck.** Problem, solution, market, product, traction, model, team, ask; one message per slide.

Tools and resources (official sites, not checked live)

- PowerPoint help
- Google Slides help

ASSIGNMENT

Practice for lesson 7.1

Business problem: Rebuild a cluttered chart slide.

Steps

1. Remove gridlines and chart junk.
2. Highlight the key bar.
3. Write a finding as the title.

Expected output: A slide understood in five seconds.

SOLUTION (ANSWER KEY)

A title such as 'Revenue doubled in Q3' beats 'Revenue chart'.

Check yourself

1. What belongs in a chart title?
   Answer: The finding.
2. Why use a slide master?
   Answer: Consistency.

8. Branding Designer

Lesson 8.1: Full brand identity systems

What you learn

- **Style guide.** Logo rules, colours, type, imagery, spacing and examples of right and wrong use.
- **Usage guidelines.** Clear space, minimum size and misuse examples.
- **Packaging mockups.** Show the identity on real formats; label mockups as concepts.
- **Brand voice.** Three traits with do and don't phrases.

Tools and resources (official sites, not checked live)

- Brand guidelines examples

ASSIGNMENT

Practice for lesson 8.1

Business problem: Write a four-page style guide.

Steps

1. Page 1 logo.
2. Page 2 colour and type.
3. Page 3 imagery.
4. Page 4 voice.

Expected output: A guide a stranger could follow.

SOLUTION (ANSWER KEY)

Have someone unfamiliar with the project apply it to a new poster to test it.

Check yourself

1. What is the test of a style guide?
   Answer: A stranger can use it correctly.
2. What are mockups for?
   Answer: To show the identity in use.

9. 3D Modeler

Lesson 9.1: Polygon modelling, PBR texturing, lighting and rendering in Blender

What you learn

- **Polygon modelling.** Build shapes from vertices, edges and faces; keep clean quads.
- **PBR materials.** Base colour, roughness, metallic and normal maps describe how a surface reacts to light.
- **Studio lighting.** Key, fill and rim lights.
- **Rendering.** Choose Cycles for realism or Eevee for speed; denoise and set samples to balance time and quality.

Tools and resources (official sites, not checked live)

- Blender manual

ASSIGNMENT

Practice for lesson 9.1

Business problem: Model and render a mug.

Steps

1. Model the mug.
2. Apply a ceramic material.
3. Light it with three lights.
4. Render at two sample counts.

Expected output: A rendered mug and a render-time note.

SOLUTION (ANSWER KEY)

Compare the two renders to find the lowest samples that still look clean.

Check yourself

1. What does roughness control?
   Answer: How blurry the reflections are.
2. What is a key light?
   Answer: The main light.

DEPARTMENT 4

Marketing

Lessons for Department 4. Each role has numbered lessons with what you learn, official tools, a practice task with an answer, and short questions. The matching skill textbook is the Master Freelancing Marketing book.

1. Social Media Manager

Lesson 1.1: Daily workflow and operations routine

What you learn

- **Content calendar.** Plan topics, formats, dates and owners; leave room for news.
- **Community engagement.** Reply within a set time and escalate complaints.
- **Analytics.** Track reach, engagement and clicks in Metricool or Buffer; review weekly and monthly.

Tools and resources (official sites, not checked live)

- Metricool
- Buffer

ASSIGNMENT

Practice for lesson 1.1

Business problem: Plan one week.

Steps

1. List five posts with goals.
2. Schedule them.
3. Note the metrics you will check.

Expected output: A one-week calendar with metrics.

SOLUTION (ANSWER KEY)

Attach one goal to each post, such as clicks or replies.

Check yourself

1. What is a calendar for?
   Answer: Planning and consistency.
2. When do you review analytics?
   Answer: Weekly and monthly.

Lesson 1.2: Account onboarding and audit procedures

What you learn

- **Onboarding.** Collect logins safely (use platform roles, not shared passwords), brand assets, goals, audience and approvals.
- **Audit.** Review profile, posting frequency, top posts, audience and competitors.
- **Report.** Findings, three quick wins and a 30-day plan.

Tools and resources (official sites, not checked live)

- Meta Business Help

ASSIGNMENT

Practice for lesson 1.2

Business problem: Audit one account.

Steps

1. Score profile, content and engagement.
2. List three quick wins.
3. Write a 30-day plan.

Expected output: A two-page audit.

SOLUTION (ANSWER KEY)

Use the same scorecard for every client so audits are comparable.

Check yourself

1. How should you get client access?
   Answer: Through roles and permissions, not shared passwords.
2. What does an audit end with?
   Answer: A plan.

2. Google Ads Expert

Lesson 2.1: Keyword research, ad groups and match types

What you learn

- **Intent.** Searches show intent: informational, commercial or transactional. Spend on commercial and transactional first.
- **Ad groups.** One theme per group so ads match searches.
- **Match types.** Exact matches the same meaning, phrase matches searches that include the meaning in order, broad matches a wider set of related searches. Use negative keywords to block waste.

Tools and resources (official sites, not checked live)

- Google Ads Help

ASSIGNMENT

Practice for lesson 2.1

Business problem: Build three ad groups.

Steps

1. Collect 30 keywords.
2. Group by theme.
3. Choose match types and 10 negatives.

Expected output: A structured account plan.

SOLUTION (ANSWER KEY)

Start tight with exact and phrase, then widen using search-term data.

Check yourself

1. What does exact match do?
   Answer: Matches the same meaning as the keyword.
2. Why add negatives?
   Answer: To stop wasted spend.

Lesson 2.2: Conversion tracking, Performance Max and bidding

What you learn

- **Google Tag Manager.** Fire a conversion tag when a form is submitted; test with preview mode.
- **Performance Max.** An automated campaign type across Google channels; it needs good assets, accurate conversions and patience.
- **Bidding.** Target CPA aims for a cost per conversion; Target ROAS aims for return on ad spend. Use them after enough conversions are recorded.

Tools and resources (official sites, not checked live)

- Tag Manager
- Performance Max

ASSIGNMENT

Practice for lesson 2.2

Business problem: Set up a conversion.

Steps

1. Create the tag.
2. Test in preview.
3. Confirm one test conversion in Google Ads.

Expected output: A verified conversion.

SOLUTION (ANSWER KEY)

Never trust bidding on unverified conversions.

Check yourself

1. What do you need before Target CPA?
   Answer: Reliable conversion data.
2. How do you test a tag?
   Answer: In preview mode.

3. SEO Expert

Lesson 3.1: On-page optimisation

What you learn

- **Headers and titles.** One clear H1; titles and descriptions that match the content.
- **Schema markup.** Structured data (JSON-LD) that describes the page, such as Product or FAQ, so search engines can understand it.
- **Content.** Answer the question the searcher asked.

Tools and resources (official sites, not checked live)

- Search Central

ASSIGNMENT

Practice for lesson 3.1

Business problem: Optimise one page.

Steps

1. Fix title and description.
2. Add structured data.
3. Validate it.

Expected output: A page that validates.

SOLUTION (ANSWER KEY)

Use Google's testing tools to validate markup.

Check yourself

1. What does schema do?
   Answer: Describes the page to search engines.
2. How many H1 tags?
   Answer: One clear one.

Lesson 3.2: Technical audits, link building and Search Console

What you learn

- **Screaming Frog.** Crawls a site to find broken links, redirect chains, missing titles and duplicates.
- **Link building.** Earn links with useful content and real relationships; do not buy links.
- **Search Console.** Shows indexing problems, queries and pages; submit sitemaps.

Tools and resources (official sites, not checked live)

- Screaming Frog
- Search Console

ASSIGNMENT

Practice for lesson 3.2

Business problem: Audit a small site.

Steps

1. Crawl it.
2. List the top ten fixes.
3. Fix one and recheck.

Expected output: An audit with priorities.

SOLUTION (ANSWER KEY)

Rank fixes by impact and effort.

Check yourself

1. Should you buy links?
   Answer: No.
2. Where do you see indexing issues?
   Answer: Search Console.

4. Facebook and Meta Ads Specialist

Lesson 4.1: Campaign set-up for real estate, tech agencies and e-commerce

What you learn

- **Objectives.** Leads for real estate and agencies; sales for e-commerce.
- **Special categories.** Housing, credit, employment and some other categories have special ad rules limiting targeting. Read Meta's current policy.
- **Creative and offer.** One clear offer per ad and a matching landing page.

Tools and resources (official sites, not checked live)

- Meta ads policies

ASSIGNMENT

Practice for lesson 4.1

Business problem: Plan three campaigns.

Steps

1. Pick the objective for each.
2. Note policy limits.
3. Write one ad each.

Expected output: A plan with policy notes.

SOLUTION (ANSWER KEY)

Real estate ads need extra attention to the housing category rules.

Check yourself

1. Why check ad policies?
   Answer: Rules limit targeting for some categories.
2. What is a good first test?
   Answer: One offer, two creatives.

Lesson 4.2: Budget optimisation and ROAS: audiences and the Conversions API

What you learn

- **Custom audiences.** People who already engaged with you, using consented data.
- **Lookalike audiences.** New people similar to a source audience.
- **Conversions API (CAPI).** Sends events from your server to Meta, which helps measurement when browser tracking is blocked; set it up with consent and avoid duplicate events using event IDs.
- **ROAS.** Revenue divided by ad spend.

Tools and resources (official sites, not checked live)

- Conversions API

ASSIGNMENT

Practice for lesson 4.2

Business problem: Calculate ROAS.

Steps

1. Take spend 10,000 and revenue 35,000.
2. Compute ROAS.
3. Decide whether to scale.

Expected output: ROAS 3.5, and a decision with reasons.

SOLUTION (ANSWER KEY)

ROAS = 35,000 / 10,000 = 3.5. Compare it with the break-even ROAS from your margin before scaling.

Check yourself

1. What does CAPI add?
   Answer: Server-side event sending.
2. How do you avoid double counting?
   Answer: Use event IDs.

5. Email Marketer

Lesson 5.1: Deliverability: SPF, DKIM and DMARC

What you learn

- **SPF.** A DNS record that lists the servers allowed to send for your domain.
- **DKIM.** A signature added to each email, checked against a public key in DNS.
- **DMARC.** A policy that says what to do when SPF or DKIM fails and where to send reports. Start with p=none to monitor, then move to quarantine or reject.
- **Honest sending.** These records prove who you are; they do not make unwanted email acceptable. Mail people who asked for it and include a working unsubscribe.

Tools and resources (official sites, not checked live)

- Google sender guidelines
- DMARC overview

ASSIGNMENT

Practice for lesson 5.1

Business problem: Check a DMARC record.

Steps

1. Run the tested parser.
2. Try a record with a bad policy.
3. Write the correct record for your domain.

Expected output: A valid DMARC record with a report address.

SOLUTION (ANSWER KEY)

PYTHON

```
def parse_dmarc(txt):
    parts = [p.strip() for p in txt.split(";") if p.strip()]
    tags = dict(p.split("=", 1) for p in parts if "=" in p)
    issues = []
    if tags.get("v") != "DMARC1": issues.append("must start with v=DMARC1")
```

```
    if tags.get("p") not in ("none", "quarantine", "reject"): issues.append("p= must be none,
quarantine or reject")
    if "rua" not in tags: issues.append("add rua= to receive reports")
    return tags, issues
```

Example record (use your own domain): `v=DMARC1; p=none; rua=mailto:dmarc@example.com`.

Check yourself

1. What does DMARC p=none do?
   Answer: Monitors without blocking.
2. What does DKIM add?
   Answer: A verifiable signature.

Lesson 5.2: Outreach infrastructure at scale, safely

What you learn

- **Tools.** Instantly.ai and Smartlead manage multiple mailboxes, warm-up, sequences and reply tracking.
- **Rules before volume.** Contact only relevant recipients, follow the anti-spam and privacy laws of your and their country, identify yourself honestly, include an opt-out and honour it at once.
- **Safe practice.** Use a separate sending domain, authenticate it, start with small volumes, watch bounces and complaints, and stop if they rise.
- **Job applications.** Mass-emailing employers rarely works. Fewer, tailored, well-researched applications get better results.
- **Do not.** Buy lists, fake sender names or rotate domains to dodge blocks.

Tools and resources (official sites, not checked live)

- Instantly
- Smartlead

ASSIGNMENT

Practice for lesson 5.2

Business problem: Design a 50-contact pilot.

Steps

1. Choose a relevant list you can justify.
2. Write a two-step sequence with opt-out.
3. Set stop rules for bounces and complaints.

Expected output: A pilot plan with rules and limits.

SOLUTION (ANSWER KEY)

Stop and review if bounces or complaints rise. Never scale a campaign that annoys people.

Check yourself

1. Why use a separate sending domain?
   Answer: To protect your main domain's reputation.
2. What must every message include?
   Answer: An honest sender and an opt-out.

6. Marketing Automation Consultant

Lesson 6.1: CRM integration, lead scoring and nurture funnels

What you learn

- **CRM integration.** Connect forms, ads and the CRM (HubSpot or GoHighLevel) so each lead has a source.
- **Lead scoring.** Add points for fit (role, size) and behaviour (pricing page visits); pass leads to sales at a set score.
- **Nurture.** Send helpful emails matched to the stage.

Tools and resources (official sites, not checked live)

- HubSpot Academy
- GoHighLevel help

ASSIGNMENT

Practice for lesson 6.1

Business problem: Design a scoring model.

Steps

1. List five actions with points.
2. Set a threshold.
3. Test on ten sample leads.

Expected output: A model and a test table.

SOLUTION (ANSWER KEY)

Review results with sales and adjust the points.

Check yourself

1. Why record lead source?
   Answer: To see which channel works.
2. Who sets the threshold?
   Answer: Marketing and sales together.

7. Lead Generation Specialist

Lesson 7.1: B2B prospecting, outreach scripts and PPC landing pages

What you learn

- **Sourcing.** Apollo.io and LinkedIn Sales Navigator help find businesses and roles; follow each platform's terms and privacy law, and do not scrape what the terms forbid.
- **Outreach scripts.** Short, relevant, with one clear question.
- **PPC landing pages.** One offer, a matching headline, proof, a short form and fast loading.

Tools and resources (official sites, not checked live)

- Apollo
- Sales Navigator

ASSIGNMENT

Practice for lesson 7.1

Business problem: Build a 100-prospect list and one landing page.

Steps

1. Define the ideal customer.
2. Source and verify the list lawfully.
3. Write the script and page.

Expected output: A list, script and page.

SOLUTION (ANSWER KEY)

Quality of fit matters more than the size of the list.

Check yourself

1. What is the first step of prospecting?
   Answer: Define the ideal customer.
2. What must you follow when using these platforms?
   Answer: Their terms and privacy law.

DEPARTMENT 5

Data Analytics and Data Science

Lessons for Department 5. Each role has numbered lessons with what you learn, official tools, a practice task with an answer, and short questions. The matching skill textbook is the Master Freelancing Data Analytics and Data Science book.

1. Data Analytics and AI Tools

Lesson 1.1: AI-powered analytics with Microsoft Copilot

What you learn

- **Copilot in Excel.** Ask questions in natural language to summarise data, suggest charts and spot trends; availability depends on your Microsoft plan.
- **Prompt used.** 'Summarize quarterly growth trends and highlight the top 5 outlier product categories by margin percentage.'
- **Verify.** Check any AI result against the data; the code below shows how to find outliers yourself with the IQR rule.

Tools and resources (official sites, not checked live)

- Copilot in Excel

ASSIGNMENT

Practice for lesson 1.1

Business problem: Find margin outliers two ways.

Steps

1. Run the prompt in Copilot if available.
2. Run the tested code.
3. Compare the answers.

Expected output: The same outlier product from both methods.

SOLUTION (ANSWER KEY)

PYTHON

```
import pandas as pd
df = pd.DataFrame({"product":["A","B","C","D","E","F"],"revenue":
[100,120,90,110,95,900],"cost":[60,70,50,65,55,600]})
df["margin_pct"] = ((df["revenue"]-df["cost"])/df["revenue"]*100).round(1)
q1, q3 = df["margin_pct"].quantile([0.25, 0.75]); iqr = q3-q1
outliers = df[(df["margin_pct"] < q1-1.5*iqr) | (df["margin_pct"] > q3+1.5*iqr)]
print(df.describe().loc[["mean","min","max"]]); print("outliers:",
outliers["product"].tolist())
assert df["margin_pct"].iloc[0] == 40.0
```

Product F has a 33.3 percent margin while the others are near 40 percent, so the rule flags it.

Check yourself

1. Why verify AI analysis?
   Answer: It can be wrong.
2. What is the IQR rule?
   Answer: Outliers lie beyond 1.5 times the interquartile range from the quartiles.

2. Data Scientist

Lesson 2.1: EDA with Python, feature engineering and predictive models

What you learn

- **EDA.** Look at shape, types, missing values, distributions and relationships using Pandas, Matplotlib and Seaborn.
- **Feature engineering.** Create useful inputs such as margin percentage or days since last purchase.
- **Predictive models.** Train on a split and measure on unseen data; see the machine learning lesson in Department 2.

Tools and resources (official sites, not checked live)

- Seaborn
- Matplotlib

ASSIGNMENT

Practice for lesson 2.1

Business problem: Create a feature.

Steps

1. Run the EDA code above.
2. Add a new feature column.
3. Explain why it may help a model.

Expected output: A new column and a reason.

SOLUTION (ANSWER KEY)

Margin percentage is a feature because it combines revenue and cost into one comparable number.

Check yourself

1. What is a feature?
   Answer: An input column for a model.
2. What does EDA reveal?
   Answer: Data quality and patterns.

3. Data Engineer

Lesson 3.1: ETL pipelines, data warehousing and indexing

What you learn

- **ETL.** Extract the raw data, transform it (clean, dedupe, type), load it into a table. The tested code below does all three.
- **Warehouses.** BigQuery and Snowflake store large data for analysis; they separate storage from compute and charge differently, so check current pricing.
- **Indexing.** An index speeds lookups on a column but slows writes slightly.

Tools and resources (official sites, not checked live)

- BigQuery
- Snowflake

ASSIGNMENT

Practice for lesson 3.1

Business problem: Run a tiny ETL.

Steps

1. Run the tested pipeline.
2. Add a bad row.
3. Check it is dropped.

Expected output: A clean table with one total per city.

SOLUTION (ANSWER KEY)

PYTHON

```
import sqlite3, csv, io
raw = "order_id,city,amount\n1, pune ,500\n2,Delhi,\n3,PUNE,300\n3,PUNE,300\n"
def extract(text): return list(csv.DictReader(io.StringIO(text)))
def transform(rows):
    seen, out = set(), []
    for r in rows:
        if not r["amount"].strip(): continue              # drop rows with a missing amount
        key = r["order_id"]
        if key in seen: continue                          # drop duplicate order ids
        seen.add(key); out.append((int(key), r["city"].strip().title(), int(r["amount"])))
    return out
def load(rows, db=":memory:"):
    con = sqlite3.connect(db); con.execute("create table orders(order_id int primary key, city
text, amount int)")
    con.executemany("insert into orders values (?,?,?)", rows); con.execute("create index
idx_city on orders(city)"); return con
```

The blank-amount row and the duplicate order are removed, and 'pune' becomes 'Pune'.

Check yourself

1. What does the transform step do?
   Answer: Cleans and reshapes data.

2. What does an index cost?
   Answer: Slightly slower writes.

4. Power BI Specialist

Lesson 4.1: Data modelling, star schemas, DAX and publishing

What you learn

- **Star schema.** A central fact table (sales) linked to dimension tables (date, product, customer).
- **DAX.** CALCULATE changes the filter context of a measure. SAMEPERIODLASTYEAR returns the same dates in the previous year, so you can compare with it.
- **Publishing.** Publish to the Power BI Service, set refresh and share by workspace with correct access.

Tools and resources (official sites, not checked live)

- Power BI documentation

ASSIGNMENT

Practice for lesson 4.1

Business problem: Write a year-on-year measure.

Steps

1. Create Total Sales = SUM(Sales[Amount]).
2. Create Sales LY = CALCULATE([Total Sales], SAMEPERIODLASTYEAR('Date'[Date])).
3. Create the growth percentage.

Expected output: A card showing growth versus last year.

SOLUTION (ANSWER KEY)

Sales LY needs a proper date table marked as the date table. Growth % = DIVIDE([Total Sales] - [Sales LY], [Sales LY]).

Check yourself

1. What is a fact table?
   Answer: The central table of events such as sales.
2. What does SAMEPERIODLASTYEAR need?
   Answer: A continuous date table.

DEPARTMENT 6

Admin and Business Support

Lessons for Department 6. Each role has numbered lessons with what you learn, official tools, a practice task with an answer, and short questions. The matching skill textbook is the Master Freelancing Admin and Business Support book.

1. Virtual Assistant

Lesson 1.1: Executive support fundamentals

What you learn

- **Inbox zero.** Process email to empty using four actions: do, delegate, defer, delete.
- **Calendar across time zones.** Store events in the right zone, confirm in the guest's local time and add buffers.
- **Travel planning.** Build an itinerary with times, confirmation numbers, transfers and a backup; keep documents in one folder.

Tools and resources (official sites, not checked live)

- Google Calendar help

ASSIGNMENT

Practice for lesson 1.1

Business problem: Schedule a three-zone meeting.

Steps

1. Pick a time that suits India, the UK and the US East coast.
2. Write the invitation in each local time.
3. Add a 10-minute buffer.

Expected output: A reasonable time and clear invitations.

SOLUTION (ANSWER KEY)

The overlap is narrow, so compare a few options and give the client the best one.

Check yourself

1. What are the four inbox actions?
   Answer: Do, delegate, defer, delete.
2. Why state local times?
   Answer: To avoid mistakes.

Lesson 1.2: Remote SOP creation in Notion

What you learn

- **SOP structure.** Purpose, owner, tools, numbered steps, checks and what to do when it fails.
- **Notion.** Create a template database so every SOP has the same fields.
- **Test.** Ask someone to follow it without help.

Tools and resources (official sites, not checked live)

- Notion help

ASSIGNMENT

Practice for lesson 1.2

Business problem: Write one SOP.

Steps

1. Pick a task.
2. Write numbered steps.
3. Have someone test it.

Expected output: An SOP a stranger can follow.

SOLUTION (ANSWER KEY)

Fix every step the tester asked about.

Check yourself

1. What is a good SOP test?
   Answer: A stranger completes it unaided.
2. Why a template?
   Answer: Consistency.

2. Data Entry Specialist

Lesson 2.1: Advanced data cleansing, verification and OCR

What you learn

- **Cleansing.** Trim spaces, fix case, standardise dates and phone numbers, remove duplicates.
- **Verification controls.** Validate fields, double-check key values and sample your own work.
- **OCR.** Convert scanned PDFs to text, then proofread; OCR makes errors on poor scans.

Tools and resources (official sites, not checked live)

- OpenRefine

ASSIGNMENT

Practice for lesson 2.1

Business problem: Validate records.

Steps

1. Run the tested validator.
2. Add two bad rows.
3. Report the error rate.

Expected output: A list of errors and a rate.

SOLUTION (ANSWER KEY)

PYTHON

```
import re

def clean_phone(raw):
    digits = re.sub(r"\D", "", raw)
    if digits.startswith("91") and len(digits) == 12:
        digits = digits[2:]
    return digits if len(digits) == 10 else None

def valid_email(raw):
    return bool(re.fullmatch(r"[^@\s]+@[^@\s]+\.[^@\s]+", raw.strip()))

def validate_record(rec):
    errors = []
    if not rec.get("name", "").strip():
        errors.append("name missing")
    if not valid_email(rec.get("email", "")):
        errors.append("email invalid")
    if clean_phone(rec.get("phone", "")) is None:
        errors.append("phone must be 10 digits (after removing +91)")
    return errors

def reconcile(bank, ledger):
    """Match bank lines to ledger entries by amount and date. Returns matched, unmatched_bank,
unmatched_ledger."""
    remaining = list(ledger)
    matched, unmatched_bank = [], []
    for b in bank:
        hit = next((l for l in remaining if l["amount"] == b["amount"] and l["date"] ==
b["date"]), None)
        if hit:
            matched.append((b["id"], hit["id"])); remaining.remove(hit)
        else:
            unmatched_bank.append(b["id"])
    return matched, unmatched_bank, [l["id"] for l in remaining]
```

Each bad record returns its problems, so errors are fixed from the source, not guessed.

Check yourself

1. Why proofread OCR output?
   Answer: OCR makes errors.

2. What is a verification control?
   Answer: A check that catches mistakes.

3. Bookkeeper and QuickBooks Consultant

Lesson 3.1: Double-entry accounting, chart of accounts, reconciliation and ageing

What you learn

- **Double entry.** Every transaction has equal debits and credits.
- **Chart of accounts.** A list of categories for assets, liabilities, income and expenses.
- **Bank reconciliation.** Match bank lines with the ledger and investigate unmatched items; the code is tested in the Admin textbook.
- **Ageing reports.** Accounts receivable and payable ageing show what is overdue by 30, 60 and 90 days in QuickBooks Online.
- **Boundary.** Bookkeeping records transactions; tax advice belongs to a qualified professional.

Tools and resources (official sites, not checked live)

- QuickBooks

ASSIGNMENT

Practice for lesson 3.1

Business problem: Reconcile a month.

Steps

1. Use the matcher in the Admin textbook.
2. List unmatched items.
3. Write a note for the accountant.

Expected output: Matched and unmatched lists.

SOLUTION (ANSWER KEY)

The matcher pairs by date and amount; unmatched items need explanations.

Check yourself

1. What is a debit and credit rule?
   Answer: They are equal for each entry.
2. What does ageing show?
   Answer: How overdue amounts are.

4. Cold Caller and Appointment Setter

Lesson 4.1: Scripting, gatekeepers, objections and calendar booking

What you learn

- **Compliance first.** Check Do Not Disturb and calling-hour rules where you call; log and honour opt-outs.
- **Scripting.** Open with who you are and why, ask permission, ask one question, offer a time.
- **Gatekeepers.** Be polite and honest; ask for help finding the right person.
- **Objections.** Acknowledge, ask, answer briefly, offer a next step.
- **Booking.** Offer two times and send a calendar invite.

Tools and resources (official sites, not checked live)

- Calendly

ASSIGNMENT

Practice for lesson 4.1

Business problem: Write and role-play a script.

Steps

1. Write the opening and two questions.
2. Write answers to three objections.
3. Role-play with a friend.

Expected output: A one-page script and a call log.

SOLUTION (ANSWER KEY)

Record each outcome so you can improve the weakest step.

Check yourself

1. What comes first, before calling?
   Answer: The compliance check.
2. What do you offer at the end?
   Answer: Two meeting times.

DEPARTMENT 7

Writing and Content

Lessons for Department 7. Each role has numbered lessons with what you learn, official tools, a practice task with an answer, and short questions. The matching skill textbook is the Master Freelancing Writing and Content book.

1. Resume Writer

Lesson 1.1: Industry-specific ATS optimisation

What you learn

- **Fintech and tech resumes.** Metric-driven bullets, a technical skills block and the exact keywords from the job posting, used honestly where they apply.
- **Keyword mapping.** List the posting's skills and tools and check each appears where it is true. Do not stuff keywords or hide white text.
- **Creative and content resumes.** Use a clean structure that applicant systems read (plain headings, no text in images) and show visual style in a linked portfolio.
- **Honesty.** Use only facts and numbers the client confirms.

Tools and resources (official sites, not checked live)

- Jobscan guides

ASSIGNMENT

Practice for lesson 1.1

Business problem: Check bullets.

Steps

1. Run the tested bullet checker.
2. Rewrite any flagged bullet.
3. Map keywords from one job post.

Expected output: Bullets that pass and a keyword map.

SOLUTION (ANSWER KEY)

PYTHON

```
import re
def word_count(t): return len(re.findall(r"[A-Za-z0-9']+", t))
def reading_ease(t):
    """Flesch reading ease (approximate syllable count)."""
    words=re.findall(r"[A-Za-z']+", t); sents=max(1,len(re.findall(r"[.!?]+", t)))
    def syl(w):
        w=w.lower(); n=len(re.findall(r"[aeiouy]+", w))
        if w.endswith("e") and n>1: n-=1
        return max(1,n)
    s=sum(syl(w) for w in words)
    return round(206.835-1.015*(len(words)/sents)-84.6*(s/max(1,len(words))),1)
def passive_hits(t): return re.findall(r"\b(?:was|were|is|are|been|being)\s+\w+ed\b", t)
```

```
def check_resume_bullet(b):
    issues=[]
    if not re.search(r"\d", b): issues.append("no number or result")
    if re.match(r"^(responsible for|worked on|helped)", b.lower()): issues.append("weak
opening")
    if word_count(b)>28: issues.append("too long")
    return issues
```

A bullet without a number or with a weak opening is flagged for rewriting.

Check yourself

1. Why not keyword-stuff?
   Answer: It reads badly and can be rejected.
2. Where do creative resumes show visuals?
   Answer: In a linked portfolio, not in image text.

2. Copywriter

Lesson 2.1: Direct response copy, headlines and funnel scripts (PAS and AIDA)

What you learn

- **PAS.** Problem, Agitate, Solution: name the problem, show what it costs, offer the fix.
- **AIDA.** Attention, Interest, Desire, Action.
- **Headlines.** Specific beats clever: name the customer and the result.
- **Claims.** Every claim must be true and provable.

Tools and resources (official sites, not checked live)

- Hemingway Editor

ASSIGNMENT

Practice for lesson 2.1

Business problem: Write a PAS landing page.

Steps

1. Write the problem.
2. Agitate in two sentences.
3. Present the solution and one call to action.

Expected output: A one-screen page.

SOLUTION (ANSWER KEY)

Problem: late invoices. Agitate: cash flow stalls. Solution: automatic reminders. Action: start a free trial.

Check yourself

1. What does PAS stand for?
   Answer: Problem, Agitate, Solution.
2. What makes a headline strong?
   Answer: Specificity.

3. Technical Writer

Lesson 3.1: Developer documentation, API references and knowledge base guides in Markdown

What you learn

- **API reference.** For each endpoint: method, URL, parameters, example request, example response and errors.
- **Knowledge base guides.** One task per article with numbered steps and expected results.
- **Markdown.** Plain text with simple marks, easy to version in Git.
- **Test.** Run every example yourself.

Tools and resources (official sites, not checked live)

- Markdown guide

ASSIGNMENT

Practice for lesson 3.1

Business problem: Document one endpoint.

Steps

1. Describe GET /users/1.
2. Add a request and a response.
3. List error codes.

Expected output: A tested reference page.

SOLUTION (ANSWER KEY)

Use the tested server from Department 1, lesson 1.1, to make sure the example response is real.

Check yourself

1. What must every API entry show?
   Answer: Request, response and errors.
2. Why Markdown?
   Answer: Easy to version and edit.

4. Scriptwriter and Ghostwriter

Lesson 4.1: VSL scripts, YouTube outlines and e-book ghostwriting

What you learn

- **Video sales letter.** Hook, problem, story, solution, proof, offer, guarantee (only a true one) and call to action.
- **YouTube educational outline.** Hook in 15 seconds, promise, three sections, recap and next step.
- **E-book ghostwriting.** Interview the client, outline chapters, write in their voice, revise with them and agree credit and ownership in writing.
- **Timing.** About 130 spoken words per minute at an easy pace.

Tools and resources (official sites, not checked live)

- Otter

ASSIGNMENT

Practice for lesson 4.1

Business problem: Outline a six-minute video.

Steps

1. Write the hook.
2. Outline three sections.
3. Count words to fit the time.

Expected output: An outline of about 780 words.

SOLUTION (ANSWER KEY)

Six minutes at 130 words a minute is about 780 words.

Check yourself

1. How many words fit in a minute?
   Answer: About 130.
2. What must be agreed before ghostwriting?
   Answer: Credit and ownership.

Master AI video prompting system

The prompt structure and production-ready examples for AI video.

1. The 6-element video prompt formula

To get consistent results while using few render credits, build every prompt in this order:

TEXT

```
[Subject & Motion] + [Environment/Setting] + [Lighting & Mood] + [Camera Movement] +
[Style/Renderer] + [Technical Parameters]
```

USE THE RIGHT PARAMETERS

`--ar 16:9` and similar flags are Midjourney syntax. In Runway, Kling and Pika, set the aspect ratio, duration and camera in the tool's own controls and remove the flag from the text. Resolution words such as 8k or 24fps are style hints, not guaranteed settings. The mythological prompt depicts a religious figure: keep it respectful and check the tool's content rules and your audience before publishing.

2. Production-ready prompts by genre

Scene genre 1: cartoon animation

Tool target: Kling AI / Runway Gen-3

TEXT

```
3D animated render of a friendly young female inventor with round glasses adjusting a tiny copper
robot on a wooden workbench, warm golden ambient lighting, detailed workshop background filled
with glowing gears, slow push-in camera shot, Pixar animation style, 4k resolution, 24fps --ar
16:9
```

Scene genre 2: mythological or historical narrative (Ramayan theme)

Tool target: Kling AI, or Midjourney (v6 image) then Runway Gen-3 (motion)

TEXT

```
Cinematic wide shot of an ancient Indian warrior prince holding a glowing golden bow on a cliff
edge overlooking a tempestuous ocean, stormy skies illuminated by distant lightning, dramatic
cinematic lighting, wind blowing through embroidered silk robes, dynamic tracking shot, Unreal
Engine 5 render, hyper-detailed, 8k --ar 16:9
```

Scene genre 3: dynamic action or fight scene

Tool target: Runway Gen-3 / Pika Labs

TEXT

```
High-speed action tracking shot of a cyberpunk martial artist dodging a strike on a wet neon-lit
rooftop in heavy rain, slow motion impact effect, dynamic camera orbit, neon blue and magenta rim
lighting, cinematic motion blur, 80mm lens shot, anamorphic lens flare --ar 16:9
```

Scene genre 4: multi-character dramatic scene

Tool target: Kling AI / Runway Gen-3

TEXT

```
Wide shot inside a medieval stone throne room: a crowned King in fur-trimmed robes conversing with a Queen, while a hooded villain in black armor stands in shadow near the grand archway, atmospheric haze, dramatic chiaroscuro lighting, slow panning camera, movie screenshot --ar 16:9
```

AI integration and tool workflows

Workflows that connect forms, automation, AI and messaging.

1. WordPress, n8n and AI integration workflow

TEXT

```
[User Form / WooCommerce] --(Webhook)--> [n8n Workflow Engine]
                                                |
              +-------------------------+-------------------------+
              v                                                   v
   [OpenAI / Claude API]                                [Google Sheets API]
 (Generate custom response)                          (Log customer lead data)
              |                                                   |
              +-------------------------+-------------------------+
                                        v
                          [Twilio WhatsApp / Slack API]
                         (Send instant alert & reply)
```

Safeguards: send WhatsApp messages only to customers who opted in; verify the webhook signature; use the order ID to avoid duplicates; keep API keys in the n8n credentials store; add an error workflow that alerts you.

2. Prompting a language model for code

Use a fixed structure so the answer is specific and reviewable:

TEXT

```
Role: Senior Full-Stack Developer
Task: Build a Next.js Server Action for secure user authentication using Prisma ORM.
Constraints:
- Include input validation using a Zod schema.
- Implement password hashing with bcrypt.
- Return a structured JSON response with error handling.
- Do not use deprecated Next.js API routes.
```

Then review the answer: run it, test bad input, and check that no secrets or unsafe defaults were added. Use the best current model the tool offers; model names change quickly.

Tool integration directory

Where to find each tool. Free tiers and prices change; always check the official page.

Tool	Primary function	Free tier and pricing (verify)	Key use	Official link
Runway ML	Gen-2 and Gen-3 video generation	Free trial and paid plans	Text-to-video and image-to-video	runwayml.com
Kling AI	Realistic video generation	Free daily credits reported; check current	High-motion scenes	klingai.com
ElevenLabs	Voice generation and dubbing	Free plan available; check current	Voice-overs and dubbing	elevenlabs.io
n8n	Workflow automation	Open-source self-hosting and paid cloud	Custom workflows	n8n.io
Lovable	AI web app generation	Free trial and paid plans	Prompt-to-app	lovable.dev
Cursor	AI code editor	Free tier and paid plans	AI-assisted coding	cursor.com

VERIFICATION NOTE

Tools, plans, prices, model names and platform rules change often. Links point to official sites that were not checked live in this draft. Confirm every price, limit and policy on the official site before relying on it. Samples use fictional data. This book is learning material, not legal, tax or financial advice, and it does not promise jobs or income.